

Análisis y Diseño de Algoritmos I

Valeria Bengolea
Departamento de Computación
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Universidad Nacional de Río Cuarto

Introducción al Diseño de Algoritmos

La Noción de Algoritmo

Algoritmo

Es una secuencia finita de instrucciones rigurosas y bien definidas, cuyo objetivo es resolver un problema específico especificado en un contrato (ya sea de forma formal o informal).

Los algoritmos describen cálculos:

- Pueden ser implementados como **programas** en un lenguaje de programación o también pueden ser descritos de forma abstracta usando **pseudocódigo**
- El pseudocódigo es una notación informal para describir abstractamente programas
 - Se omiten las reglas sintácticas estrictas de los lenguajes de programación
 - Los tipos, operadores y estructuras de control se usan de forma flexible
 - Las subtarefas pueden referirse como funciones/procedimientos (tarefas simples o tareas complejas que se refinarán/implementarán más adelante)

Un Ejemplo: Cálculo del Máximo Común Divisor

A modo de ejemplo, podemos considerar el problema de calcular el máximo común divisor (mcd), y un algoritmo básico para resolverlo.

Problema: Dados dos números naturales positivos x e y , calcular el mayor número natural d que divida a x e y .

Algoritmo:

Paso 1. Obtener r , el mínimo entre x e y

Paso 2. Si r divide a x e y , retornar r .

Paso 3. Sino, decrementar r en 1 y volver al paso 2.

Un Ejemplo: Cálculo del Máximo Común Divisor

Como programadores, estamos acostumbrados a describir algoritmos mediante código o pseudo-código:

```
r ← MIN(x,y)
while x mod r != 0 or y mod r != 0 do
  begin
    r ← r-1
  end
return r
```


Soluciones Alternativas

Los problemas que admiten soluciones algorítmicas suelen poder resolverse de muchas formas alternativas. Por ejemplo, el problema de calcular el máximo común divisor entre dos números puede resolverse de otras maneras:

Algoritmo de Euclides

$\text{mcd}(x, y)$

$$\text{mcd}(x, 0) = x$$

$$\text{mcd}(x, y) = \text{mcd}(y, x \bmod y)$$

Soluciones Alternativas

Los problemas que admiten soluciones algorítmicas suelen poder resolverse de muchas formas alternativas. Por ejemplo, el problema de calcular el máximo común divisor entre dos números puede resolverse de otras maneras:

Algoritmo de Euclides

$\text{mcd}(x, y)$

$$\begin{aligned}\text{mcd}(x, 0) &= x \\ \text{mcd}(x, y) &= \text{mcd}(y, x \bmod y)\end{aligned}$$

Paso 1. Si $y = 0$, retornar x como resultado y terminar. Caso contrario, continuar con el paso 2.

Paso 2. Dividir x por y , obtener el resto r de la división. Continuar con el paso 3.

Paso 3. Volver al paso 1 para calcular el máximo común divisor, esta vez de y y r .

Soluciones Alternativas

Los problemas que admiten soluciones algorítmicas suelen poder resolverse de muchas formas alternativas. Por ejemplo, el problema de calcular el máximo común divisor entre dos números puede resolverse de otras maneras:

Algoritmo de Euclides

$\text{mcd}(x, y)$

Paso 1. Si $y = 0$, retornar x como resultado y terminar. Caso contrario, continuar con el paso 2.

Paso 2. Dividir x por y , obtener el resto r de la división. Continuar con el paso 3.

Paso 3. Volver al paso 1 para calcular el máximo común divisor, esta vez de y y r .

$$\begin{aligned}\text{mcd}(x, 0) &= x \\ \text{mcd}(x, y) &= \text{mcd}(y, x \bmod y)\end{aligned}$$

```
while y!=0 do
  begin
    r ← x mod y
    x ← y
    y ← r
  end
return x
```

Soluciones Alternativas

Los problemas que admiten soluciones algorítmicas suelen poder resolverse de muchas formas alternativas. Por ejemplo, el problema de calcular el máximo común divisor entre dos números puede resolverse de otras maneras:

Algoritmo de Euclides

$\text{mcd}(x, y)$

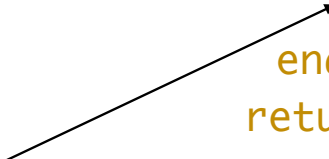
Paso 1. Si $y = 0$, retornar x como resultado y terminar. Caso contrario, continuar con el paso 2.

Paso 2. Dividir x por y , obtener el resto r de la división. Continuar con el paso 3.

Paso 3. Volver al paso 1 para calcular el máximo común divisor, esta vez de y y r .

$$\begin{aligned}\text{mcd}(x, 0) &= x \\ \text{mcd}(x, y) &= \text{mcd}(y, x \bmod y)\end{aligned}$$

```
while y!=0 do
begin
    r ← x mod y
    x ← y
    y ← r
end
return x
```



Soluciones Alternativas

Los problemas que admiten soluciones algorítmicas suelen poder resolverse de muchas formas alternativas. Por ejemplo, el problema de calcular el máximo común divisor entre dos números puede resolverse de otras maneras:

Algoritmo de Euclides

$\text{mcd}(x, y)$

Paso 1. Si $y = 0$, retornar x como resultado y terminar. Caso contrario, continuar con el paso 2.

Paso 2. Dividir x por y , obtener el resto r de la división. Continuar con el paso 3.

Paso 3. Volver al paso 1 para calcular el máximo común divisor, esta vez de y y r .

$\text{mcd}(x, 0) = x$
 $\text{mcd}(x, y) = \text{mcd}(y, x \bmod y)$

```
while y!=0 do
begin
    r ← x mod y
    x ← y
    y ← r
end
return x
```

Importante:
¿Como sabemos que termina?

Soluciones Alternativas

Los problemas que admiten soluciones algorítmicas suelen poder resolverse de muchas formas alternativas. Por ejemplo, el problema de calcular el máximo común divisor entre dos números puede resolverse de otras maneras:

Algoritmo de Euclides

$\text{mcd}(x, y)$

Paso 1. Si $y = 0$, retornar x como resultado y terminar. Caso contrario, continuar con el paso 2.

Paso 2. Dividir x por y , obtener el resto r de la división. Continuar con el paso 3.

Paso 3. Volver al paso 1 para calcular el máximo común divisor, esta vez de y y r .

$$\begin{aligned}\text{mcd}(x, 0) &= x \\ \text{mcd}(x, y) &= \text{mcd}(y, x \bmod y)\end{aligned}$$

```
while y!=0 do  
  begin
```

```
     $r \leftarrow x \bmod y$ 
```

```
     $x \leftarrow y$ 
```

```
     $y \leftarrow r$ 
```

```
  end
```

```
  return x
```

Importante:
¿Como sabemos que termina?

Como
 $0 \leq x \bmod y < y$, el valor de y disminuye estrictamente

Soluciones Alternativas

Los problemas que admiten soluciones algorítmicas suelen poder resolverse de muchas formas alternativas. Por ejemplo, el problema de calcular el máximo común divisor entre dos números puede resolverse de otras maneras:

Soluciones Alternativas

Los problemas que admiten soluciones algorítmicas suelen poder resolverse de muchas formas alternativas. Por ejemplo, el problema de calcular el máximo común divisor entre dos números puede resolverse de otras maneras:

Basado en Factorización de Enteros

Paso 1. Encontrar la factorización de x en términos de números primos.

Paso 2. Encontrar la factorización de y en términos de números primos.

Paso 3. Tomar los factores primos comunes a x e y

Paso 4. Calcular el producto de todos los factores primos comunes a x e y , y retornar esto como resultado

Soluciones Alternativas

Los problemas que admiten soluciones algorítmicas suelen poder resolverse de muchas formas alternativas. Por ejemplo, el problema de calcular el máximo común divisor entre dos números puede resolverse de otras maneras:

Basado en Factorización de Enteros

Ejemplos

Paso 1. Encontrar la factorización de x en términos de números primos.

Paso 2. Encontrar la factorización de y en términos de números primos.

Paso 3. Tomar los factores primos comunes a x e y

Paso 4. Calcular el producto de todos los factores primos comunes a x e y , y retornar esto como resultado

Soluciones Alternativas

Los problemas que admiten soluciones algorítmicas suelen poder resolverse de muchas formas alternativas. Por ejemplo, el problema de calcular el máximo común divisor entre dos números puede resolverse de otras maneras:

Basado en Factorización de Enteros

- Paso 1. Encontrar la factorización de x en términos de números primos.
- Paso 2. Encontrar la factorización de y en términos de números primos.
- Paso 3. Tomar los factores primos comunes a x e y
- Paso 4. Calcular el producto de todos los factores primos comunes a x e y , y retornar esto como resultado

Ejemplos

mcd(60, 24)

$$60 = 2 * 2 * 3 * 5$$

$$24 = 2 * 2 * 2 * 3$$

Soluciones Alternativas

Los problemas que admiten soluciones algorítmicas suelen poder resolverse de muchas formas alternativas. Por ejemplo, el problema de calcular el máximo común divisor entre dos números puede resolverse de otras maneras:

Basado en Factorización de Enteros

- Paso 1. Encontrar la factorización de x en términos de números primos.
- Paso 2. Encontrar la factorización de y en términos de números primos.
- Paso 3. Tomar los factores primos comunes a x e y
- Paso 4. Calcular el producto de todos los factores primos comunes a x e y , y retornar esto como resultado

Ejemplos

$\text{mcd}(60, 24)$

$$60 = 2 \cdot 2 \cdot 3 \cdot 5$$

$$24 = 2 \cdot 2 \cdot 2 \cdot 3$$

Soluciones Alternativas

Los problemas que admiten soluciones algorítmicas suelen poder resolverse de muchas formas alternativas. Por ejemplo, el problema de calcular el máximo común divisor entre dos números puede resolverse de otras maneras:

Basado en Factorización de Enteros

- Paso 1. Encontrar la factorización de x en términos de números primos.
- Paso 2. Encontrar la factorización de y en términos de números primos.
- Paso 3. Tomar los factores primos comunes a x e y
- Paso 4. Calcular el producto de todos los factores primos comunes a x e y , y retornar esto como resultado

Ejemplos

$$\text{mcd}(60, 24)$$

$$60 = 2 \cdot 2 \cdot 3 \cdot 5$$

$$24 = 2 \cdot 2 \cdot 2 \cdot 3$$

$$\text{mcd}(60, 24) = 2 \cdot 2 \cdot 3 = 12$$

Soluciones Alternativas

Los problemas que admiten soluciones algorítmicas suelen poder resolverse de muchas formas alternativas. Por ejemplo, el problema de calcular el máximo común divisor entre dos números puede resolverse de otras maneras:

Basado en Factorización de Enteros

- Paso 1. Encontrar la factorización de x en términos de números primos.
- Paso 2. Encontrar la factorización de y en términos de números primos.
- Paso 3. Tomar los factores primos comunes a x e y
- Paso 4. Calcular el producto de todos los factores primos comunes a x e y , y retornar esto como resultado

Ejemplos

$$\text{mcd}(60, 24)$$

$$60 = 2 \cdot 2 \cdot 3 \cdot 5$$

$$24 = 2 \cdot 2 \cdot 2 \cdot 3$$

$$\text{mcd}(60, 24) = 2 \cdot 2 \cdot 3 = 12$$

$$\text{mcd}(24, 129)$$

$$24 = 2 \cdot 2 \cdot 2 \cdot 3$$

$$129 = 3 \cdot 43$$

Soluciones Alternativas

Los problemas que admiten soluciones algorítmicas suelen poder resolverse de muchas formas alternativas. Por ejemplo, el problema de calcular el máximo común divisor entre dos números puede resolverse de otras maneras:

Basado en Factorización de Enteros

- Paso 1. Encontrar la factorización de x en términos de números primos.
- Paso 2. Encontrar la factorización de y en términos de números primos.
- Paso 3. Tomar los factores primos comunes a x e y
- Paso 4. Calcular el producto de todos los factores primos comunes a x e y , y retornar esto como resultado

Ejemplos

$$\text{mcd}(60, 24)$$

$$60 = 2 \cdot 2 \cdot 3 \cdot 5$$

$$24 = 2 \cdot 2 \cdot 2 \cdot 3$$

$$\text{mcd}(60, 24) = 2 \cdot 2 \cdot 3 = 12$$

$$\text{mcd}(24, 129)$$

$$24 = 2 \cdot 2 \cdot 2 \cdot 3$$

$$129 = 3 \cdot 43$$

Soluciones Alternativas

Los problemas que admiten soluciones algorítmicas suelen poder resolverse de muchas formas alternativas. Por ejemplo, el problema de calcular el máximo común divisor entre dos números puede resolverse de otras maneras:

Basado en Factorización de Enteros

- Paso 1. Encontrar la factorización de x en términos de números primos.
- Paso 2. Encontrar la factorización de y en términos de números primos.
- Paso 3. Tomar los factores primos comunes a x e y
- Paso 4. Calcular el producto de todos los factores primos comunes a x e y , y retornar esto como resultado

Ejemplos

$$\text{mcd}(60, 24)$$

$$60 = 2 \cdot 2 \cdot 3 \cdot 5$$

$$24 = 2 \cdot 2 \cdot 2 \cdot 3$$

$$\text{mcd}(60, 24) = 2 \cdot 2 \cdot 3 = 12$$

$$\text{mcd}(24, 129)$$

$$24 = 2 \cdot 2 \cdot 2 \cdot 3$$

$$129 = 3 \cdot 43$$

$$\text{mcd}(24, 129) = 3$$

Soluciones Alternativas

Los problemas que admiten soluciones algorítmicas suelen poder resolverse de muchas formas alternativas. Por ejemplo, el problema de calcular el máximo común divisor entre dos números puede resolverse de otras maneras:

Basado en Factorización de Enteros

¿Es ésta una descripción de un Algoritmo?

Paso 1. Encontrar la factorización de x en términos de números primos.

Paso 2. Encontrar la factorización de y en términos de números primos.

Paso 3. Tomar los factores primos comunes a x e y

Paso 4. Calcular el producto de todos los factores primos comunes a x e y, y retornar esto como resultado

Ejemplos

mcd(60, 24)

$$60 = 2 \cdot 2 \cdot 3 \cdot 5$$

$$24 = 2 \cdot 2 \cdot 2 \cdot 3$$

$$\text{mcd}(60, 24) = 2 \cdot 2 \cdot 3 = 12$$

mcd(24, 129)

$$24 = 2 \cdot 2 \cdot 2 \cdot 3$$

$$129 = 3 \cdot 43$$

$$\text{mcd}(60, 129) = 3$$

Abstracción y Refinamiento en la Descripción de Algoritmos

La descripción del proceso para computar el máximo común divisor basado en factorización de enteros constituye la descripción de un algoritmo sólo si contamos con un algoritmo para computar la factorización en términos de números primos.

Abstracción y Refinamiento en la Descripción de Algoritmos

La descripción del proceso para computar el máximo común divisor basado en factorización de enteros constituye la descripción de un algoritmo sólo si contamos con un algoritmo para computar la factorización en términos de números primos.

Constituye la descripción de un proceso abstracto, que necesita ser refinado para poder ser considerado un algoritmo.

Abstracción y Refinamiento en la Descripción de Algoritmos

Para calcular la factorización en términos de números primos de un entero, podríamos por ejemplo utilizar un algoritmo basado en la **Criba de Eratóstenes**

Transformar el proceso/algoritmo original con una descripción más detallada de las tareas que la componen se denomina refinamiento

Abstracción y Refinamiento en la Descripción de Algoritmos

Input: Entero $n \geq 2$.

Output: Lista de primos menores o iguales que n

for $p \leftarrow 2$ **to** n **do** $A[p] \leftarrow p$

for $p \leftarrow 2$ **to** $\lfloor \sqrt{n} \rfloor$ **do**

if $A[p] \neq 0$ // p no ha sido eliminado previamente de la lista

$j \leftarrow p * p$

while $j \leq n$ **do**

$A[j] \leftarrow 0$ // marcar el elemento como eliminado

$j \leftarrow j + p$

Abstracción y Refinamiento en la Descripción de Algoritmos

Ejemplo $n = 25$

Input: Entero $n \geq 2$.

Output: Lista de primos menores o iguales que n

for $p \leftarrow 2$ **to** n **do** $A[p] \leftarrow p$

for $p \leftarrow 2$ **to** $\lfloor \sqrt{n} \rfloor$ **do**

if $A[p] \neq 0$ // p no ha sido eliminado previamente de la lista

$j \leftarrow p * p$

while $j \leq n$ **do**

$A[j] \leftarrow 0$ // marcar el elemento como eliminado

$j \leftarrow j + p$

Abstracción y Refinamiento en la Descripción de Algoritmos

Ejemplo $n = 25$

Input: Entero $n \geq 2$.

Output: Lista de primos menores o iguales que n

for $p \leftarrow 2$ **to** n **do** $A[p] \leftarrow p$

for $p \leftarrow 2$ **to** $\lfloor \sqrt{n} \rfloor$ **do**

if $A[p] \neq 0$ // p no ha sido eliminado previamente de la lista

$j \leftarrow p * p$

while $j \leq n$ **do**

$A[j] \leftarrow 0$ // marcar el elemento como eliminado

$j \leftarrow j + p$

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

Abstracción y Refinamiento en la Descripción de Algoritmos

Ejemplo $n = 25$

Input: Entero $n \geq 2$.

Output: Lista de primos menores o iguales que n

for $p \leftarrow 2$ **to** n **do** $A[p] \leftarrow p$

for $p \leftarrow 2$ **to** $\lfloor \sqrt{n} \rfloor$ **do**

if $A[p] \neq 0$ // p no ha sido eliminado previamente de la lista

$j \leftarrow p * p$

while $j \leq n$ **do**

$A[j] \leftarrow 0$ // marcar el elemento como eliminado

$j \leftarrow j + p$

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
2	3	0	5	0	7	0	9	0	11	0	13	0	15	0	17	0	19	0	21	0	23	0	25

Abstracción y Refinamiento en la Descripción de Algoritmos

Ejemplo $n = 25$

Input: Entero $n \geq 2$.

Output: Lista de primos menores o iguales que n

for $p \leftarrow 2$ **to** n **do** $A[p] \leftarrow p$

for $p \leftarrow 2$ **to** $\lfloor \sqrt{n} \rfloor$ **do**

if $A[p] \neq 0$ // p no ha sido eliminado previamente de la lista

$j \leftarrow p * p$

while $j \leq n$ **do**

$A[j] \leftarrow 0$ // marcar el elemento como eliminado

$j \leftarrow j + p$

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
2	3	0	5	0	7	0	9	0	11	0	13	0	15	0	17	0	19	0	21	0	23	0	25
2	3	0	5	0	7	0	0	0	11	0	13	0	0	0	17	0	19	0	0	0	23	0	25

Abstracción y Refinamiento en la Descripción de Algoritmos

Ejemplo $n = 25$

Input: Entero $n \geq 2$.

Output: Lista de primos menores o iguales que n

for $p \leftarrow 2$ **to** n **do** $A[p] \leftarrow p$

for $p \leftarrow 2$ **to** $\lfloor \sqrt{n} \rfloor$ **do**

if $A[p] \neq 0$ // p no ha sido eliminado previamente de la lista

$j \leftarrow p * p$

while $j \leq n$ **do**

$A[j] \leftarrow 0$ // marcar el elemento como eliminado

$j \leftarrow j + p$

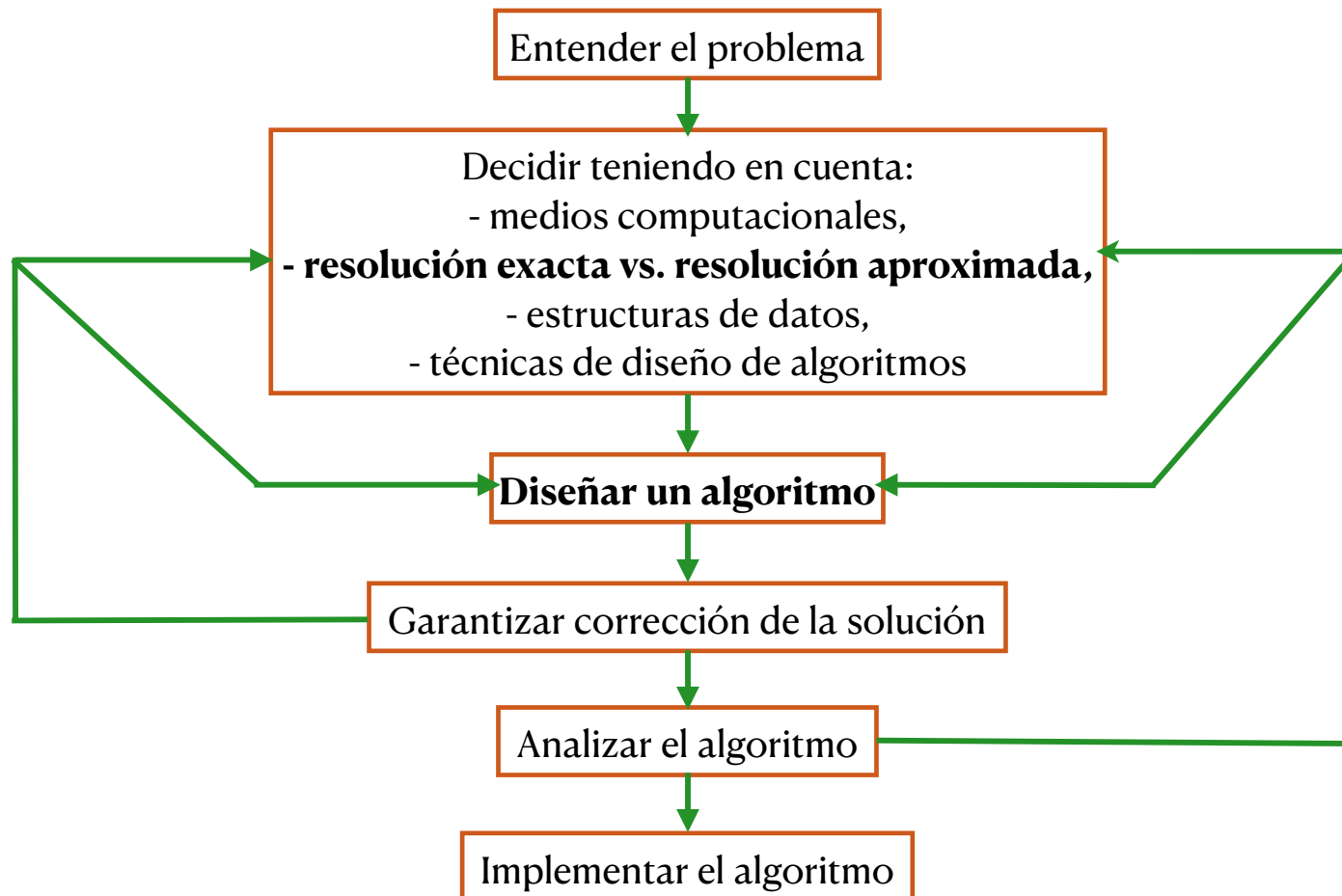
2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
2	3	0	5	0	7	0	9	0	11	0	13	0	15	0	17	0	19	0	21	0	23	0	25
2	3	0	5	0	7	0	0	0	11	0	13	0	0	0	17	0	19	0	0	0	23	0	25
2	3	0	5	0	7	0	0	0	11	0	13	0	0	0	17	0	19	0	0	0	23	0	0

Criba de Eratóstenes

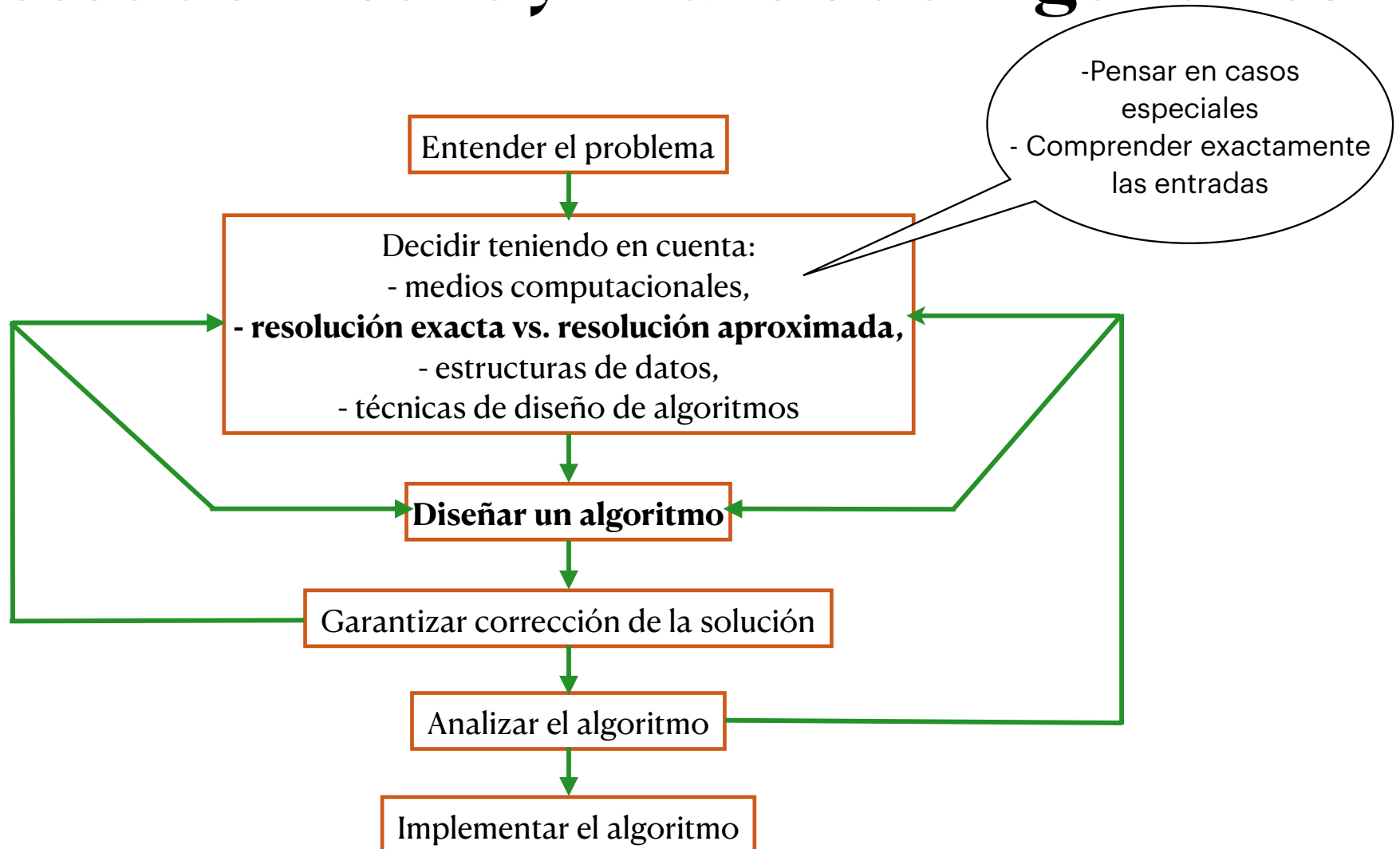
2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
2	3		5		7		9		11		13		15		17		19		21		23		25
2	3		5		7				11		13				17		19				23		25
2	3		5		7				11		13				17		19				23		

¿Cuales de estos números primos pertenecen a la factorización en números primos del 25?

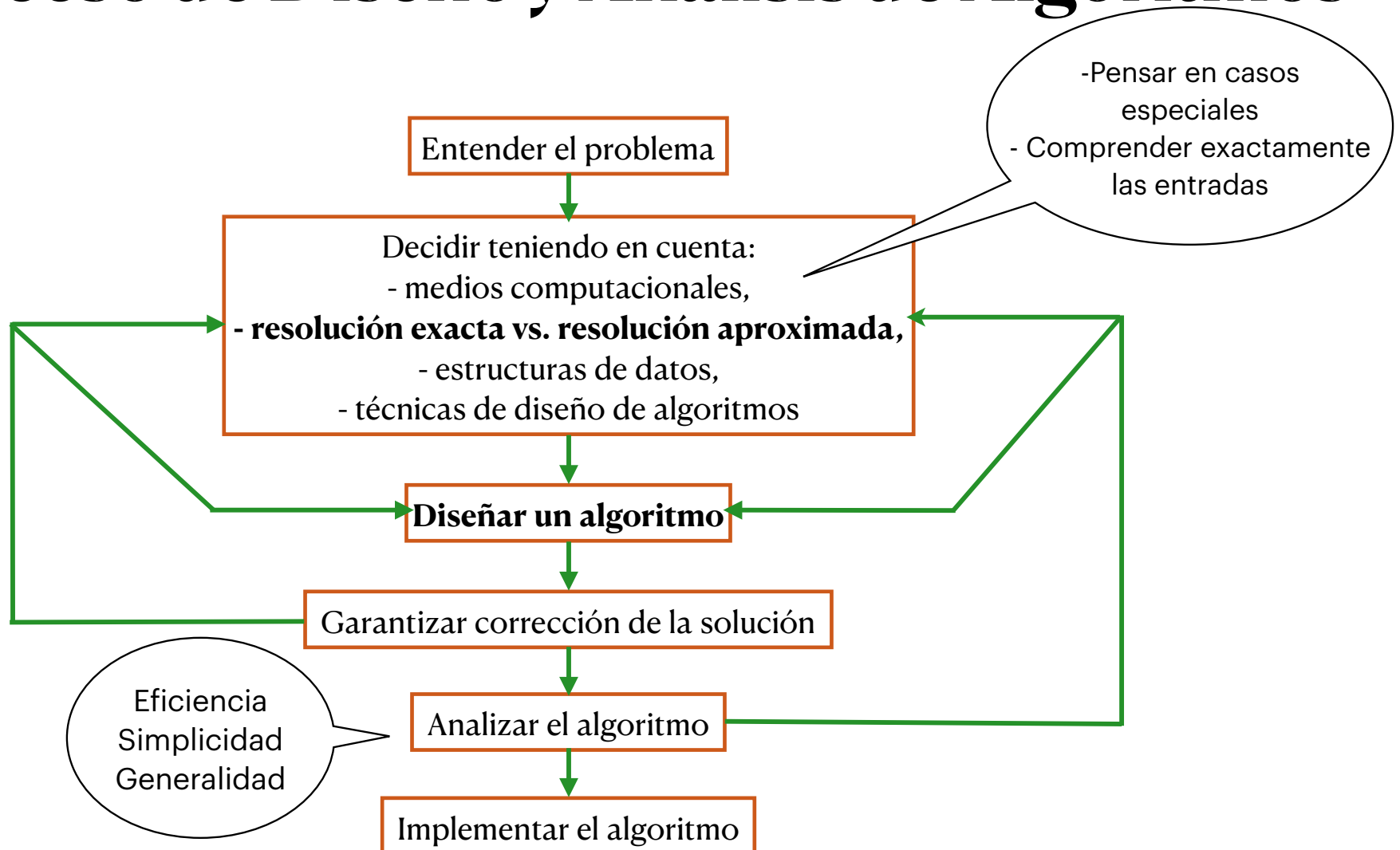
El Proceso de Diseño y Análisis de Algoritmos



El Proceso de Diseño y Análisis de Algoritmos

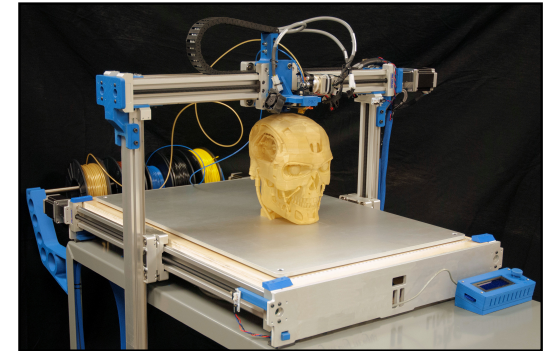


El Proceso de Diseño y Análisis de Algoritmos



Ejemplo 1:

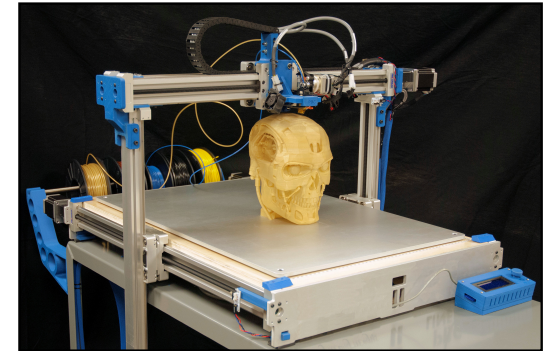
Recorrido de impresión en una impresora 3D



Ejemplo 1:

Recorrido de impresión en una impresora 3D

- Imprimen sucesivas capas 2D
 - Cada capa se imprime sobre la anterior
 - Comienza en capa $Z=0$, sigue con $Z=1$, $Z=2$, ...
 - La velocidad de impresión depende de la velocidad de impresión de cada capa
- Cada capa 2D está compuesta por un conjunto de coordenadas de impresión
 - en tales coordenadas se inyecta el material de impresión
 - velocidad de impresión depende de número de coordenadas, distancia entre ellas y recorrido elegido



Recon

- Imprimen

- Cada ca

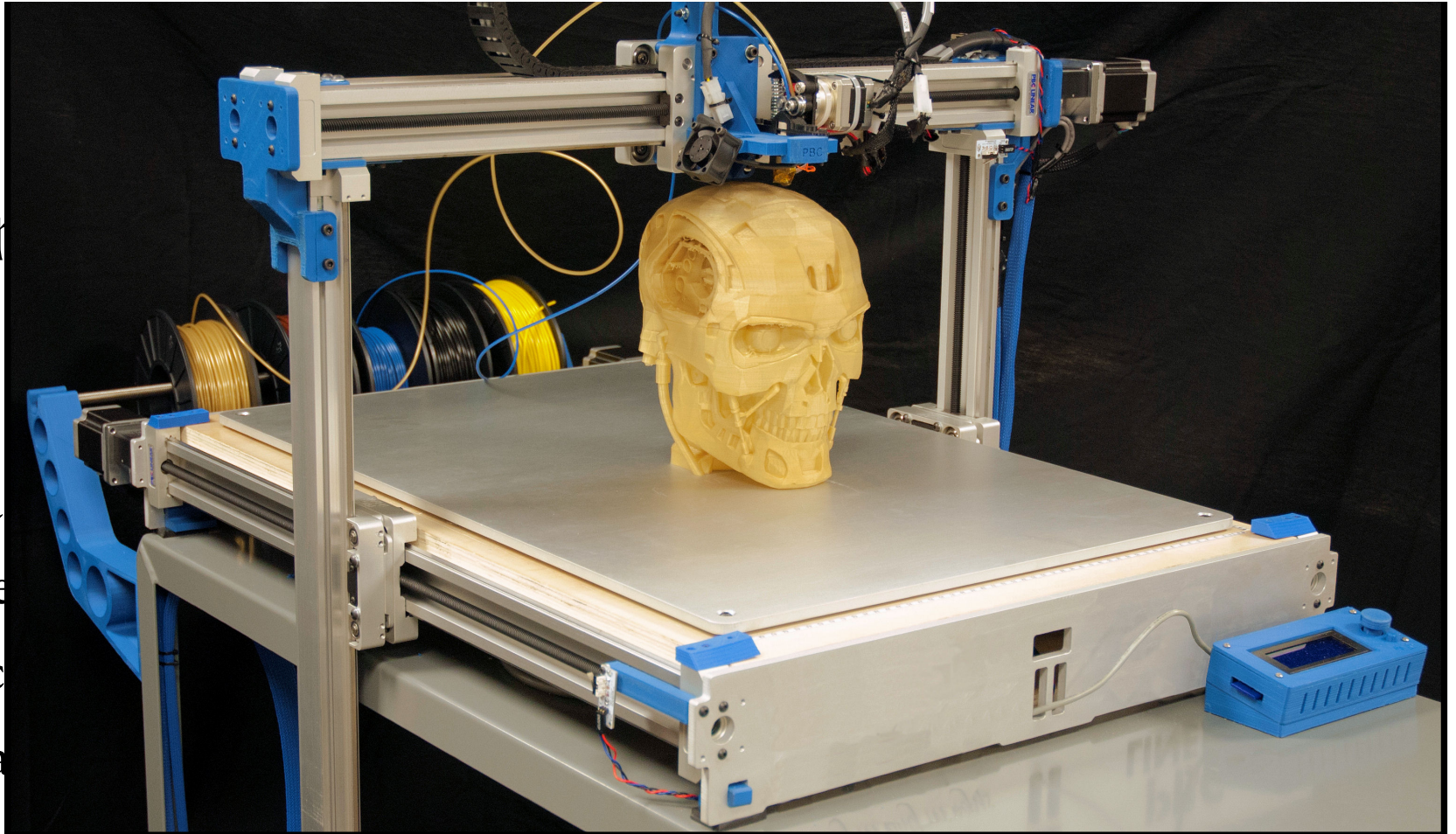
- Comie

- La veloc

- Cada capa

- en tales coordenadas se inyecta el material de impreson

- velocidad de impresión depende de número de coordenadas, distancia entre ellas y recorrido elegido



Ejemplo 1:

Recorrido de impresión en una impresora 3D

- Imprimen sucesivas capas 2D
 - Cada capa se imprime sobre la anterior
 - Comienza en capa $Z=0$, sigue con $Z=1$, $Z=2$, ...
 - La velocidad de impresión depende de la velocidad de impresión de cada capa
- Cada capa 2D está compuesta por un conjunto de coordenadas de impresión
 - en tales coordenadas se inyecta el material de impresión
 - velocidad de impresión depende de número de coordenadas, distancia entre ellas y recorrido elegido

Ejemplo:
Recorrido de impresión en una impresora 3D

Ejemplo:

Recorrido de impresión en una impresora 3D

A modo de ejemplo, ilustrando las diferentes cuestiones a tener en cuenta en la resolución algorítmica de problemas, consideremos el siguiente problema:

Ejemplo:

Recorrido de impresión en una impresora 3D

A modo de ejemplo, ilustrando las diferentes cuestiones a tener en cuenta en la resolución algorítmica de problemas, consideremos el siguiente problema:

Dado un conjunto P de puntos del plano, elegir una forma de recorrerlos de manera que la distancia global recorrida sea la mínima

Ejemplo:

Recorrido de impresión en una impresora 3D

A modo de ejemplo, ilustrando las diferentes cuestiones a tener en cuenta en la resolución algorítmica de problemas, consideremos el siguiente problema:

Dado un conjunto P de puntos del plano, elegir una forma de recorrerlos de manera que la distancia global recorrida sea la mínima

Algunas preguntas que se pueden hacer para comprender mejor el problema son las siguientes:

Ejemplo:

Recorrido de impresión en una impresora 3D

A modo de ejemplo, ilustrando las diferentes cuestiones a tener en cuenta en la resolución algorítmica de problemas, consideremos el siguiente problema:

Dado un conjunto P de puntos del plano, elegir una forma de recorrerlos de manera que la distancia global recorrida sea la mínima

Algunas preguntas que se pueden hacer para comprender mejor el problema son las siguientes:

¿hay algún punto de inicio específico?

Ejemplo:

Recorrido de impresión en una impresora 3D

A modo de ejemplo, ilustrando las diferentes cuestiones a tener en cuenta en la resolución algorítmica de problemas, consideremos el siguiente problema:

Dado un conjunto P de puntos del plano, elegir una forma de recorrerlos de manera que la distancia global recorrida sea la mínima

Algunas preguntas que se pueden hacer para comprender mejor el problema son las siguientes:

¿hay algún punto de inicio específico? **NO**

Ejemplo:

Recorrido de impresión en una impresora 3D

A modo de ejemplo, ilustrando las diferentes cuestiones a tener en cuenta en la resolución algorítmica de problemas, consideremos el siguiente problema:

Dado un conjunto P de puntos del plano, elegir una forma de recorrerlos de manera que la distancia global recorrida sea la mínima

Algunas preguntas que se pueden hacer para comprender mejor el problema son las siguientes:

¿hay algún punto de inicio específico? **NO**

¿cómo son los movimientos en el plano?

Ejemplo:

Recorrido de impresión en una impresora 3D

A modo de ejemplo, ilustrando las diferentes cuestiones a tener en cuenta en la resolución algorítmica de problemas, consideremos el siguiente problema:

Dado un conjunto P de puntos del plano, elegir una forma de recorrerlos de manera que la distancia global recorrida sea la mínima

Algunas preguntas que se pueden hacer para comprender mejor el problema son las siguientes:

¿hay algún punto de inicio específico? **NO**

¿cómo son los movimientos en el plano?

¿hay algún punto final específico?

Ejemplo:

Recorrido de impresión en una impresora 3D

A modo de ejemplo, ilustrando las diferentes cuestiones a tener en cuenta en la resolución algorítmica de problemas, consideremos el siguiente problema:

Dado un conjunto P de puntos del plano, elegir una forma de recorrerlos de manera que la distancia global recorrida sea la mínima

Algunas preguntas que se pueden hacer para comprender mejor el problema son las siguientes:

¿hay algún punto de inicio específico? **NO**

¿cómo son los movimientos en el plano? **En línea recta del origen al destino**

¿hay algún punto final específico?

Ejemplo:

Recorrido de impresión en una impresora 3D

A modo de ejemplo, ilustrando las diferentes cuestiones a tener en cuenta en la resolución algorítmica de problemas, consideremos el siguiente problema:

Dado un conjunto P de puntos del plano, elegir una forma de recorrerlos de manera que la distancia global recorrida sea la mínima

Algunas preguntas que se pueden hacer para comprender mejor el problema son las siguientes:

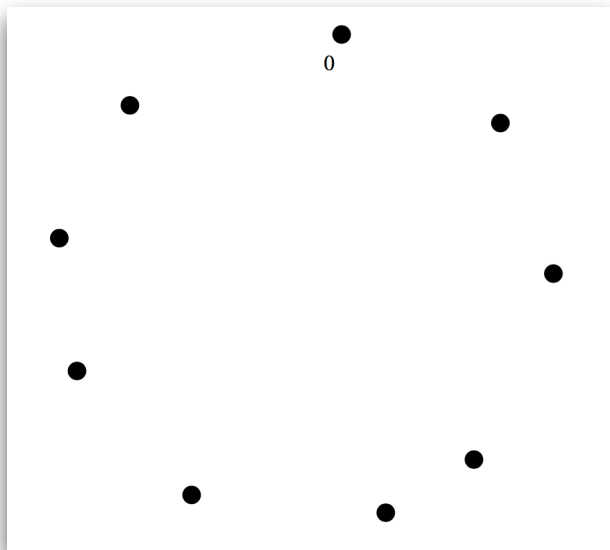
¿hay algún punto de inicio específico? **NO**

¿cómo son los movimientos en el plano? **En línea recta del origen al destino**

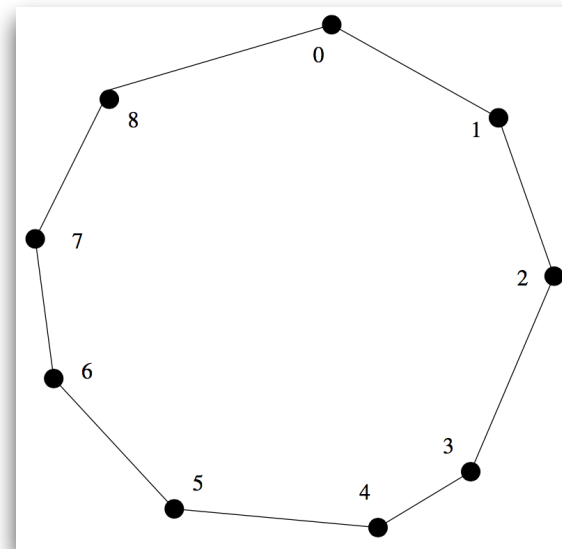
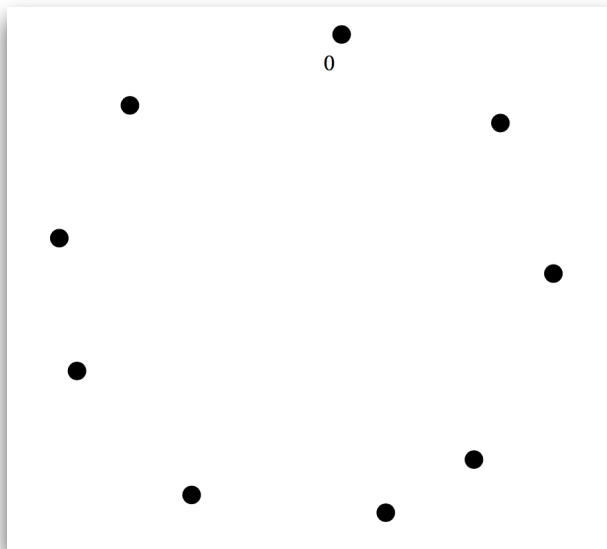
¿hay algún punto final específico? **Misma posición que punto de salida**

Ejemplos de Conjuntos de Puntos y Recorridos Esperados

Ejemplos de Conjuntos de Puntos y Recorridos Esperados

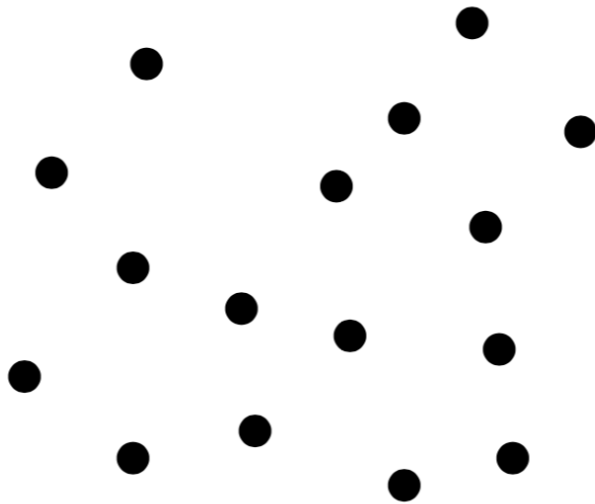


Ejemplos de Conjuntos de Puntos y Recorridos Esperados

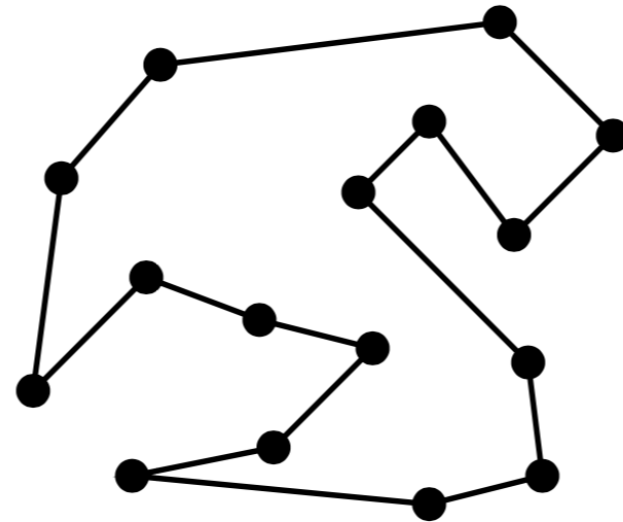
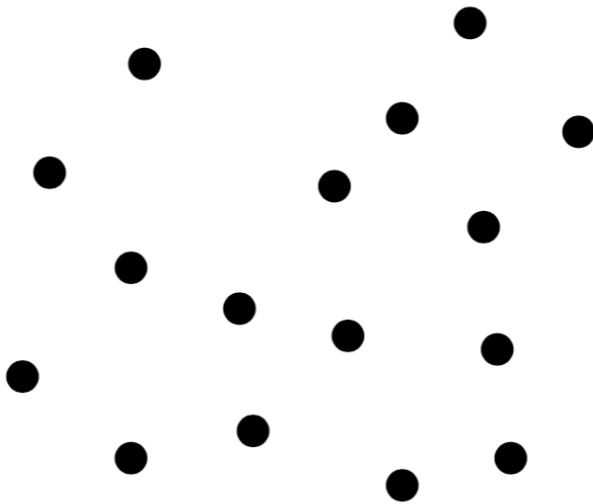


Ejemplos de Conjuntos de Puntos y Recorridos Esperados

Ejemplos de Conjuntos de Puntos y Recorridos Esperados



Ejemplos de Conjuntos de Puntos y Recorridos Esperados



Recorrido Óptimo de Impresión

Recorrido Óptimo de Impresión

Primera Propuesta

Recorrido Óptimo de Impresión

Primera Propuesta

Una propuesta para resolver el problema del recorrido óptimo de impresión es elegir siempre el punto vecino no visitado más próximo al punto actual.

Recorrido Óptimo de Impresión

Primera Propuesta

Una propuesta para resolver el problema del recorrido óptimo de impresión es elegir siempre el punto vecino no visitado más próximo al punto actual.

```
RecorridoOptimo(P: Cto. de n puntos en el plano)
  elegir un punto po de P como inicial
  visitar po
  i := 1
  while i < n do
    begin
      elegir punto pi no visitado más próximo a p(i-1)
      visitar pi
      i := i+1
    end
  retornar a po (desde p(n-1))
```

Recorrido Óptimo de Impresión

Primera Propuesta

Una propuesta para resolver el problema del recorrido óptimo de impresión es elegir siempre el punto vecino no visitado más próximo al punto actual.

```
RecorridoOptimo(P: Cto. de n puntos en el plano)
  elegir un punto po de P como inicial
  visitar po
  i := 1
  while i < n do
    begin
      elegir punto pi no visitado más próximo a p(i-1)
      visitar pi
      i := i+1
    end
  retornar a po (desde p(n-1))
```

¿Resuelve el problema
correctamente?

Recorrido Óptimo de Impresión

Primera Propuesta

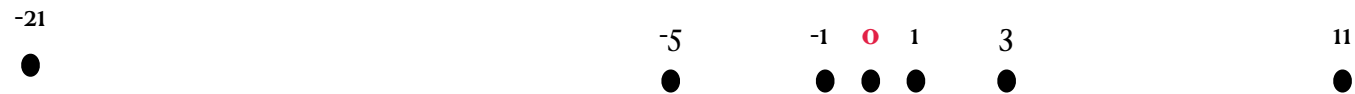
Una propuesta para resolver el problema del recorrido óptimo de impresión es elegir siempre el punto vecino no visitado más próximo al punto actual.

```
RecorridoOptimo(P: Cto. de n puntos en el plano)
  elegir un punto po de P como inicial
  visitar po
  i := 1
  while i < n do
    begin
      elegir punto pi no visitado más próximo a p(i-1)
      visitar pi
      i := i+1
    end
  retornar a po (desde p(n-1))
```

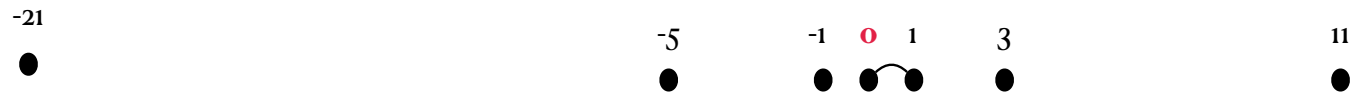
¿Resuelve el problema
correctamente?

¿Eficiencia?

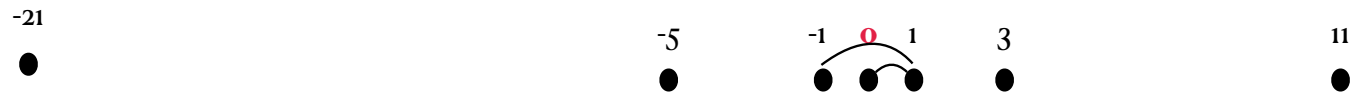
Mala Instancia de Vecino más Próximo



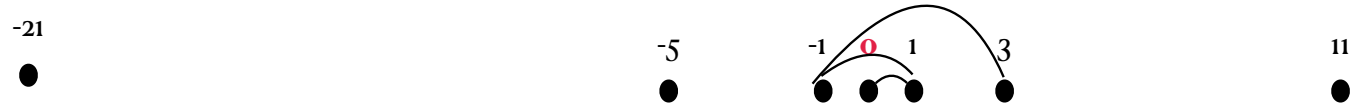
Mala Instancia de Vecino más Próximo



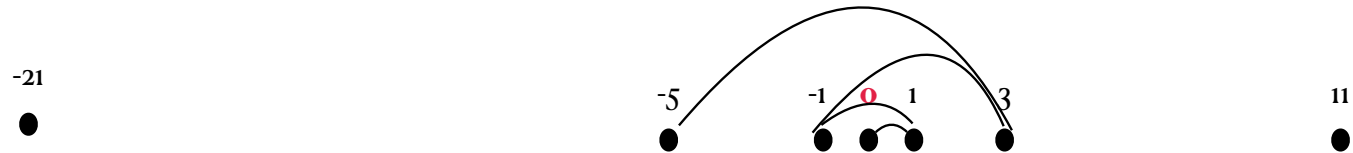
Mala Instancia de Vecino más Próximo



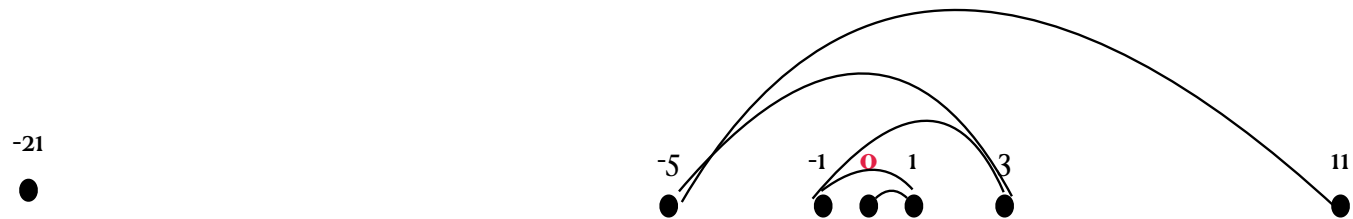
Mala Instancia de Vecino más Próximo



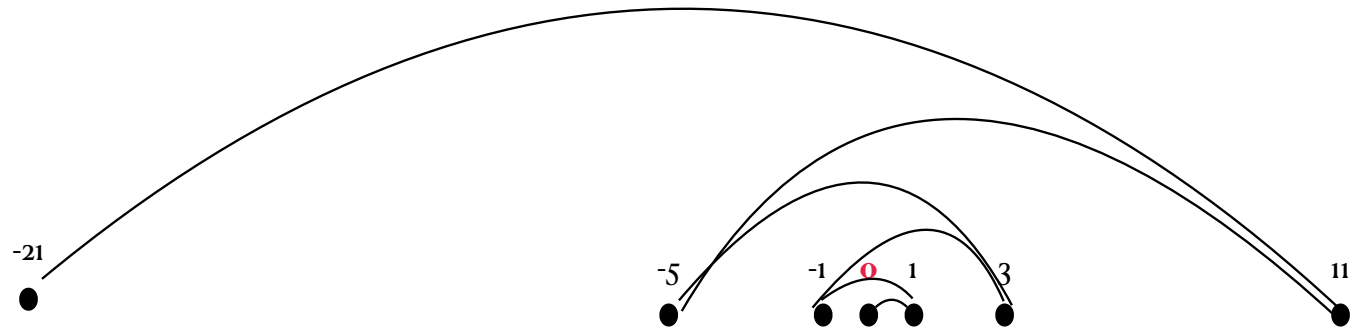
Mala Instancia de Vecino más Próximo



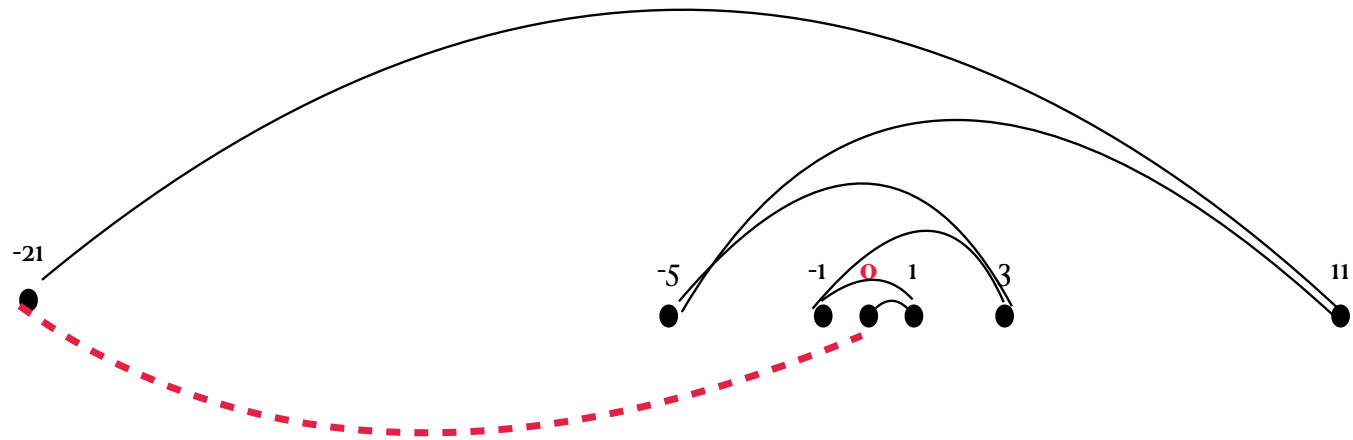
Mala Instancia de Vecino más Próximo



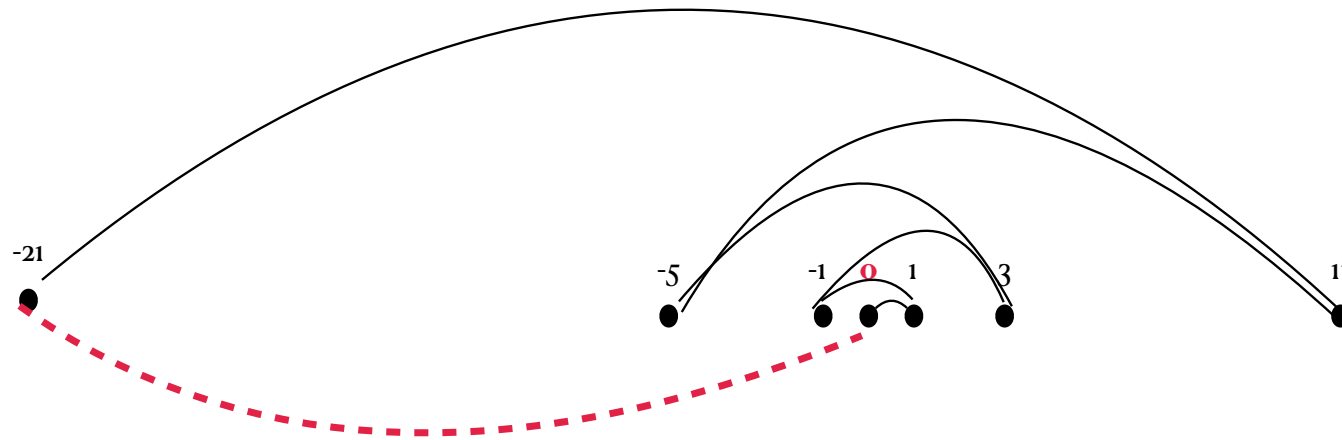
Mala Instancia de Vecino más Próximo



Mala Instancia de Vecino más Próximo

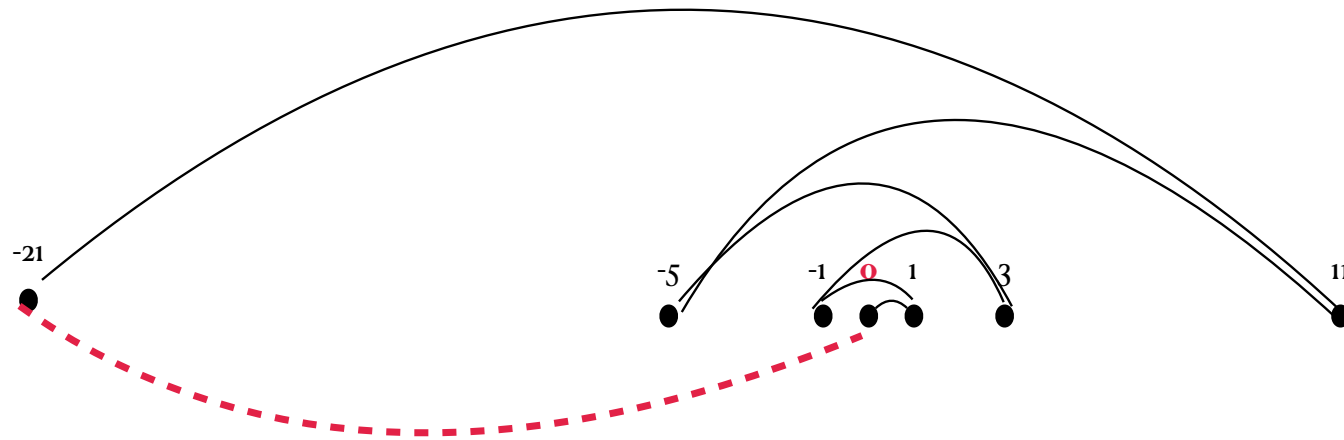


Mala Instancia de Vecino más Próximo



$$\text{Distancia} = 1 + 2 + 4 + 8 + 16 + 32 + 21 = 84$$

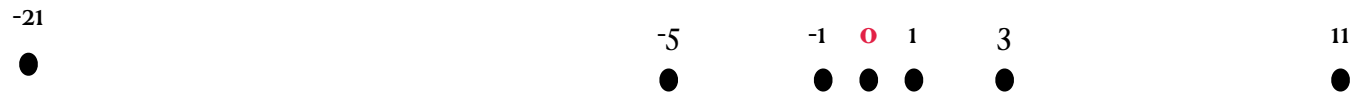
Mala Instancia de Vecino más Próximo



$$\text{Distancia} = 1 + 2 + 4 + 8 + 16 + 32 + 21 = 84$$

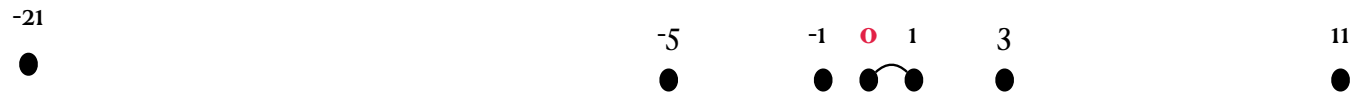
Distancia Óptima = ?

Mala Instancia de Vecino más Próximo



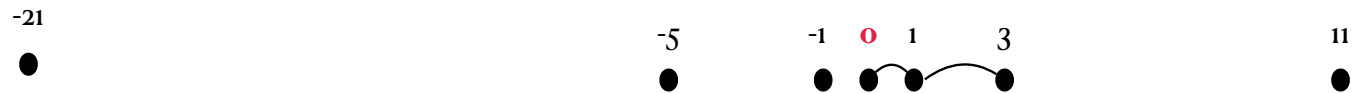
$$\text{Distancia Óptima} = 1 + 2 + 8 + 32 + 16 + 4 + 1 = \mathbf{64}$$

Mala Instancia de Vecino más Próximo



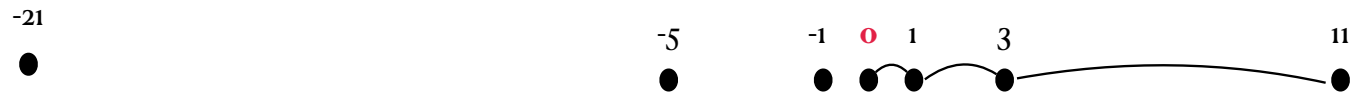
$$\text{Distancia Óptima} = 1 + 2 + 8 + 32 + 16 + 4 + 1 = \mathbf{64}$$

Mala Instancia de Vecino más Próximo



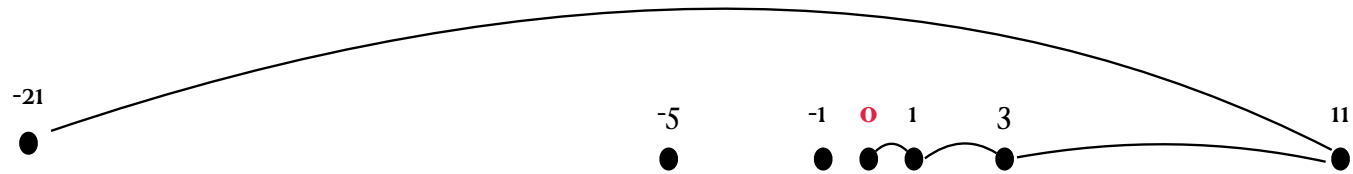
$$\text{Distancia Óptima} = 1 + 2 + 8 + 32 + 16 + 4 + 1 = \mathbf{64}$$

Mala Instancia de Vecino más Próximo



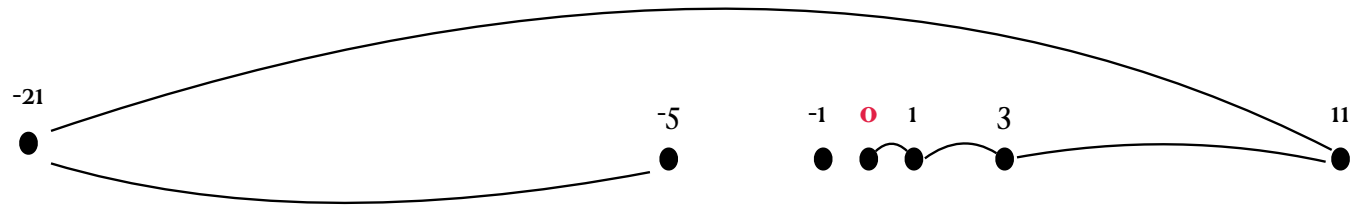
$$\text{Distancia Óptima} = 1 + 2 + 8 + 32 + 16 + 4 + 1 = \mathbf{64}$$

Mala Instancia de Vecino más Próximo



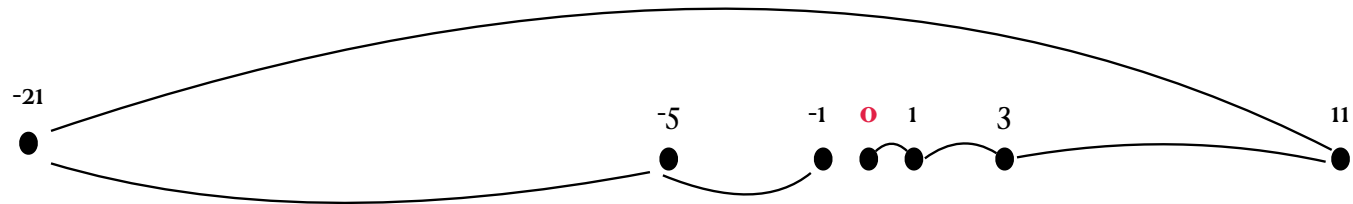
$$\text{Distancia Óptima} = 1 + 2 + 8 + 32 + 16 + 4 + 1 = \mathbf{64}$$

Mala Instancia de Vecino más Próximo



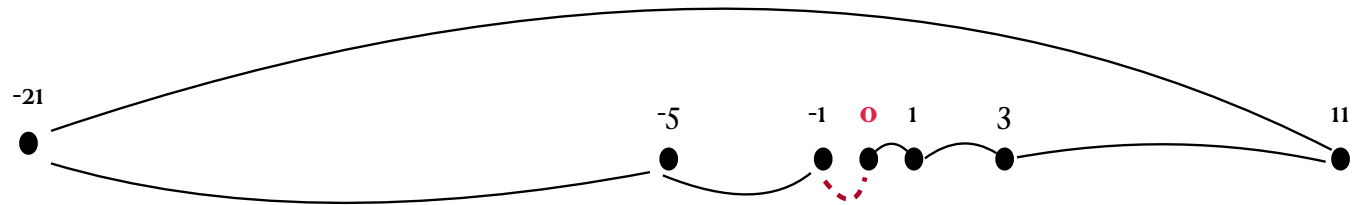
$$\text{Distancia Óptima} = 1 + 2 + 8 + 32 + 16 + 4 + 1 = \mathbf{64}$$

Mala Instancia de Vecino más Próximo



$$\text{Distancia Óptima} = 1 + 2 + 8 + 32 + 16 + 4 + 1 = \mathbf{64}$$

Mala Instancia de Vecino más Próximo



$$\text{Distancia Óptima} = 1 + 2 + 8 + 32 + 16 + 4 + 1 = \mathbf{64}$$

RecorridoOptimo'(P: Cto. de n puntos en el
plano)

for i := 1 **to** n-1 **do**

begin

d := infinity

for (s,t) en cadenas de vértices distintas **do**

begin

if dist(s,t) <= d **then**

begin

d := dist(s,t)

sm := s

tm := t

end

end

conectar sm y tm con un arco

conectar las dos puntas con un arco

RecorridoOptimo'(P: Cto. de n puntos en el
plano)

for i := 1 **to** n-1 **do**

begin

d := infinity

for (s,t) en cadenas de vértices distintas **do**

begin

if dist(s,t) <= d **then**

begin

d := dist(s,t)

sm := s

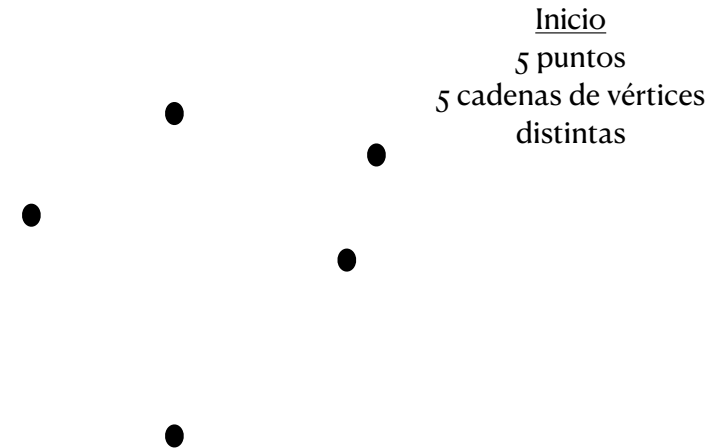
tm := t

end

end

conectar sm y tm con un arco

conectar las dos puntas con un arco



RecorridoOptimo'(P: Cto. de n puntos en el
plano)

for i := 1 **to** n-1 **do**

begin

d := infinity

for (s,t) en cadenas de vértices distintas **do**

begin

if dist(s,t) <= d **then**

begin

d := dist(s,t)

sm := s

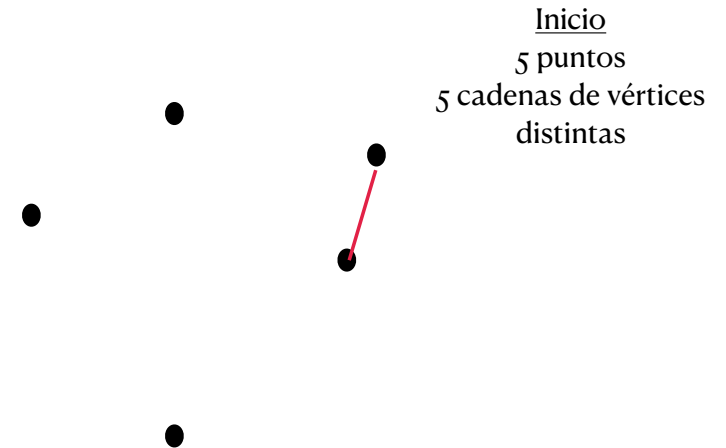
tm := t

end

end

conectar sm y tm con un arco

conectar las dos puntas con un arco



RecorridoOptimo'(P: Cto. de n puntos en el plano)

for i := 1 **to** n-1 **do**

begin

d := infinity

for (s,t) en cadenas de vértices distintas **do**

begin

if dist(s,t) <= d **then**

begin

d := dist(s,t)

sm := s

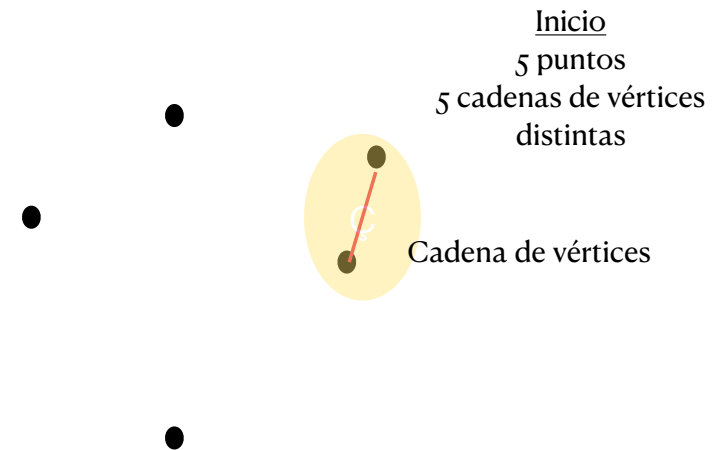
tm := t

end

end

conectar sm y tm con un arco

conectar las dos puntas con un arco



RecorridoOptimo'(P: Cto. de n puntos en el plano)

for i := 1 **to** n-1 **do**

begin

d := infinity

for (s,t) en cadenas de vértices distintas **do**

begin

if dist(s,t) <= d **then**

begin

d := dist(s,t)

sm := s

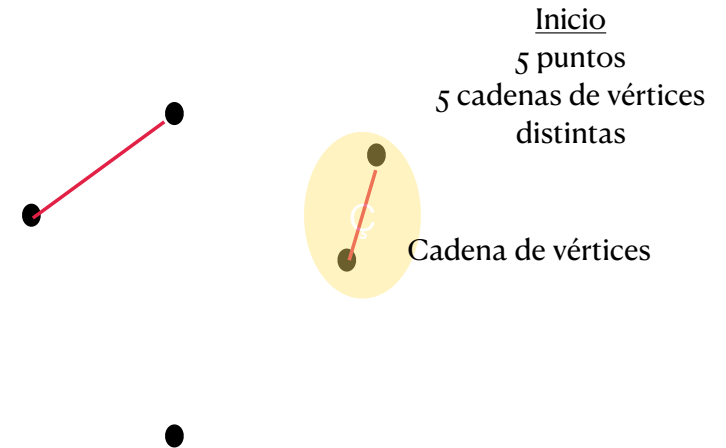
tm := t

end

end

conectar sm y tm con un arco

conectar las dos puntas con un arco



RecorridoOptimo'(P: Cto. de n puntos en el plano)

for i := 1 **to** n-1 **do**

begin

d := infinity

for (s,t) en cadenas de vértices distintas **do**

begin

if dist(s,t) <= d **then**

begin

d := dist(s,t)

sm := s

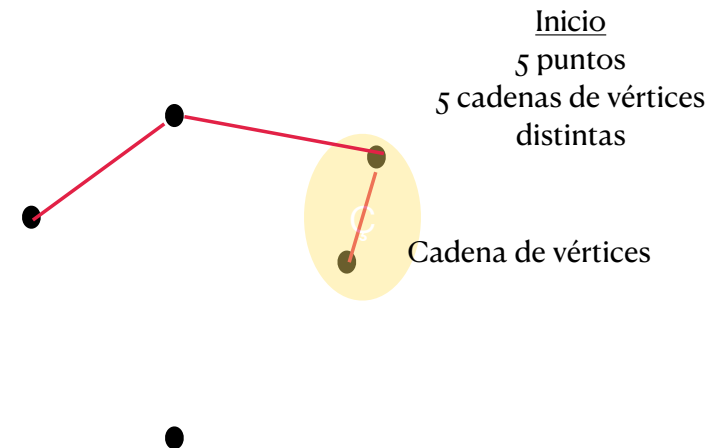
tm := t

end

end

conectar sm y tm con un arco

conectar las dos puntas con un arco



RecorridoOptimo'(P: Cto. de n puntos en el plano)

for i := 1 **to** n-1 **do**

begin

d := infinity

for (s,t) en cadenas de vértices distintas **do**

begin

if dist(s,t) <= d **then**

begin

d := dist(s,t)

sm := s

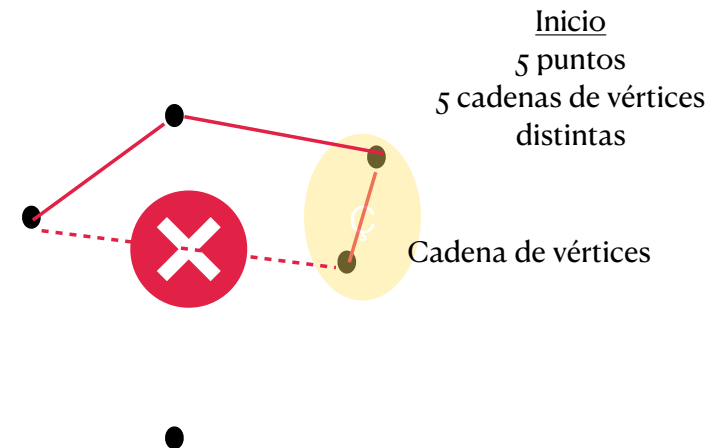
tm := t

end

end

conectar sm y tm con un arco

conectar las dos puntas con un arco



Recorrido Óptimo de Impresión

Recorrido Óptimo de Impresión

Segunda Propuesta

Recorrido Óptimo de Impresión

Segunda Propuesta

Una segunda propuesta para resolver el problema del recorrido óptimo de impresión es armar el recorrido en base a pares de puntos más próximos entre sí.

Recorrido Óptimo de Impresión

Segunda Propuesta

Una segunda propuesta para resolver el problema del recorrido óptimo de impresión es armar el recorrido en base a pares de puntos más próximos entre sí.

RecorridoOptimo'(P: Cto. de n puntos en el plano)

```
for i := 1 to n-1 do  
  begin  
    d := infinity  
    for (s,t) en cadenas de vértices distintas do  
      begin  
        if dist(s,t) <= d then  
          begin  
            d := dist(s,t)  
            sm := s  
            tm := t  
          end  
        end  
      end  
    end  
    conectar sm y tm con un arco  
    conectar las dos puntas con un arco
```

Recorrido Óptimo de Impresión

Segunda Propuesta

Una segunda propuesta para resolver el problema del recorrido óptimo de impresión es armar el recorrido en base a pares de puntos más próximos entre sí.

RecorridoOptimo'(P: Cto. de n puntos en el plano)

```
for i := 1 to n-1 do  
  begin  
    d := infinity  
    for (s,t) en cadenas de vértices distintas do  
      begin  
        if dist(s,t) <= d then  
          begin  
            d := dist(s,t)  
            sm := s  
            tm := t  
          end  
        end  
      end  
    end  
    conectar sm y tm con un arco  
  conectar las dos puntas con un arco
```

¿Resuelve el problema
correctamente?

Recorrido Óptimo de Impresión

Segunda Propuesta

Una segunda propuesta para resolver el problema del recorrido óptimo de impresión es armar el recorrido en base a pares de puntos más próximos entre sí.

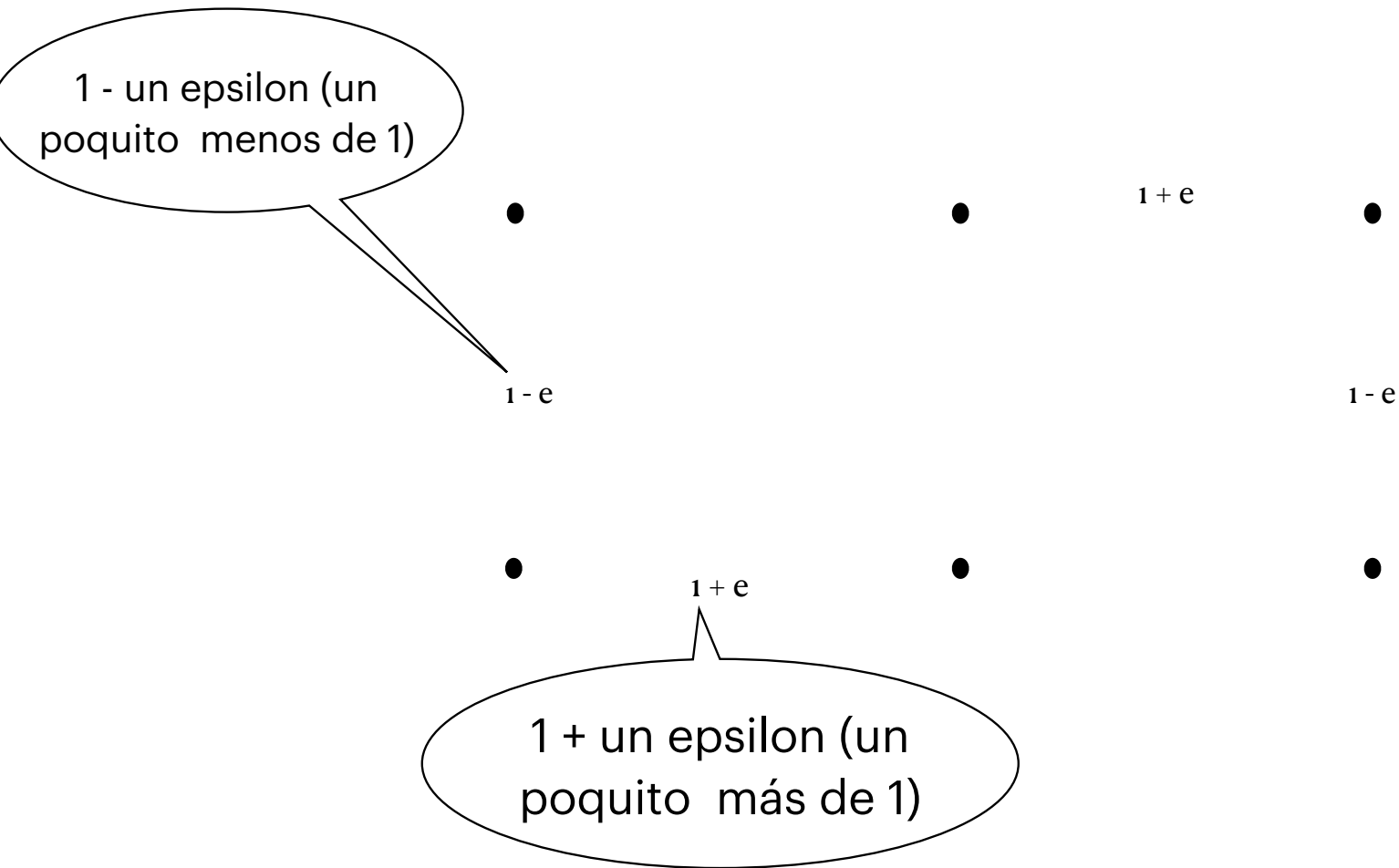
RecorridoOptimo'(P: Cto. de n puntos en el plano)

```
for i := 1 to n-1 do  
  begin  
    d := infinity  
    for (s,t) en cadenas de vértices distintas do  
      begin  
        if dist(s,t) <= d then  
          begin  
            d := dist(s,t)  
            sm := s  
            tm := t  
          end  
        end  
      end  
    end  
    conectar sm y tm con un arco  
  conectar las dos puntas con un arco
```

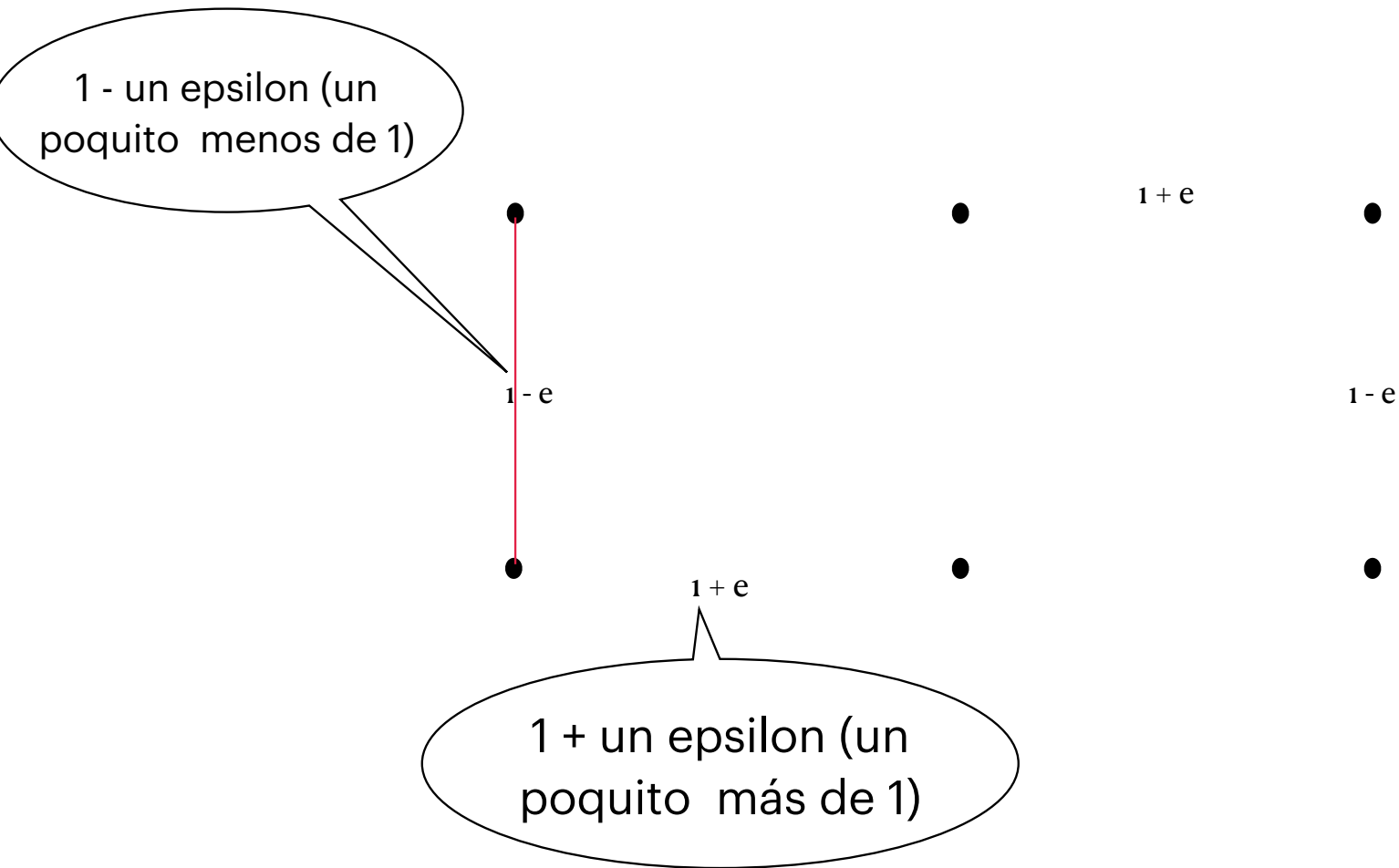
¿Resuelve el problema
correctamente?

¿Eficiencia?

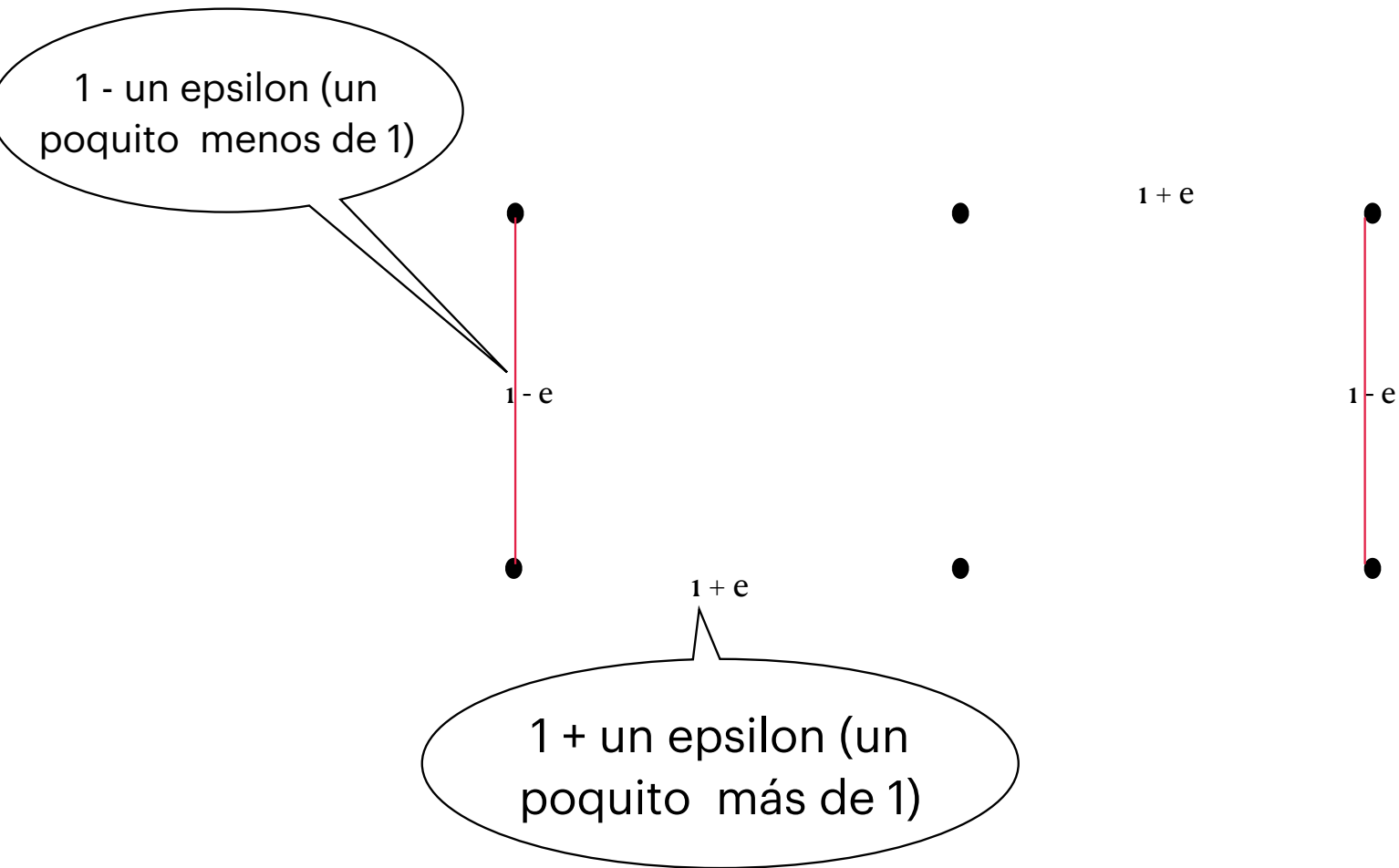
Instancia de Pares más Próximos



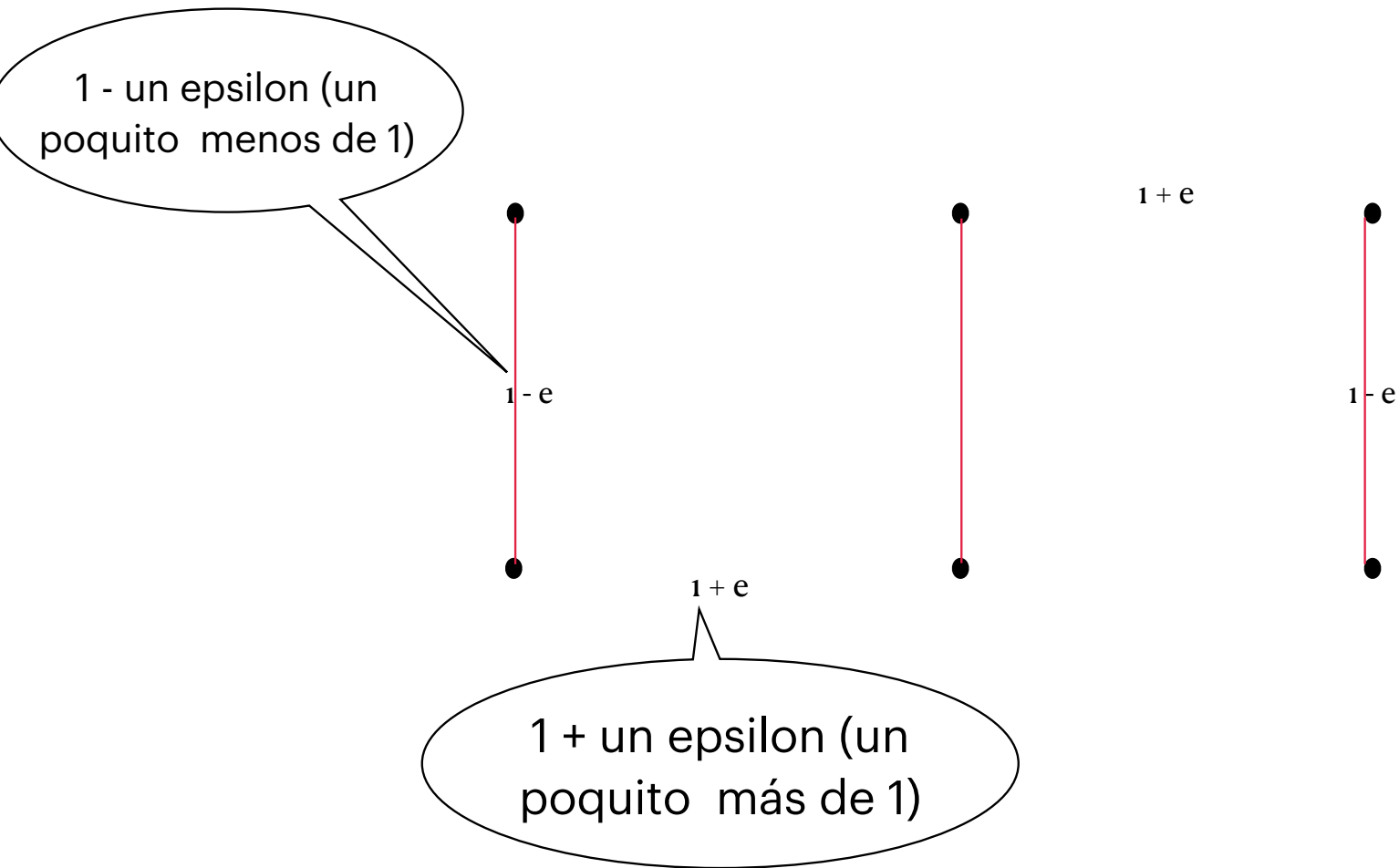
Instancia de Pares más Próximos



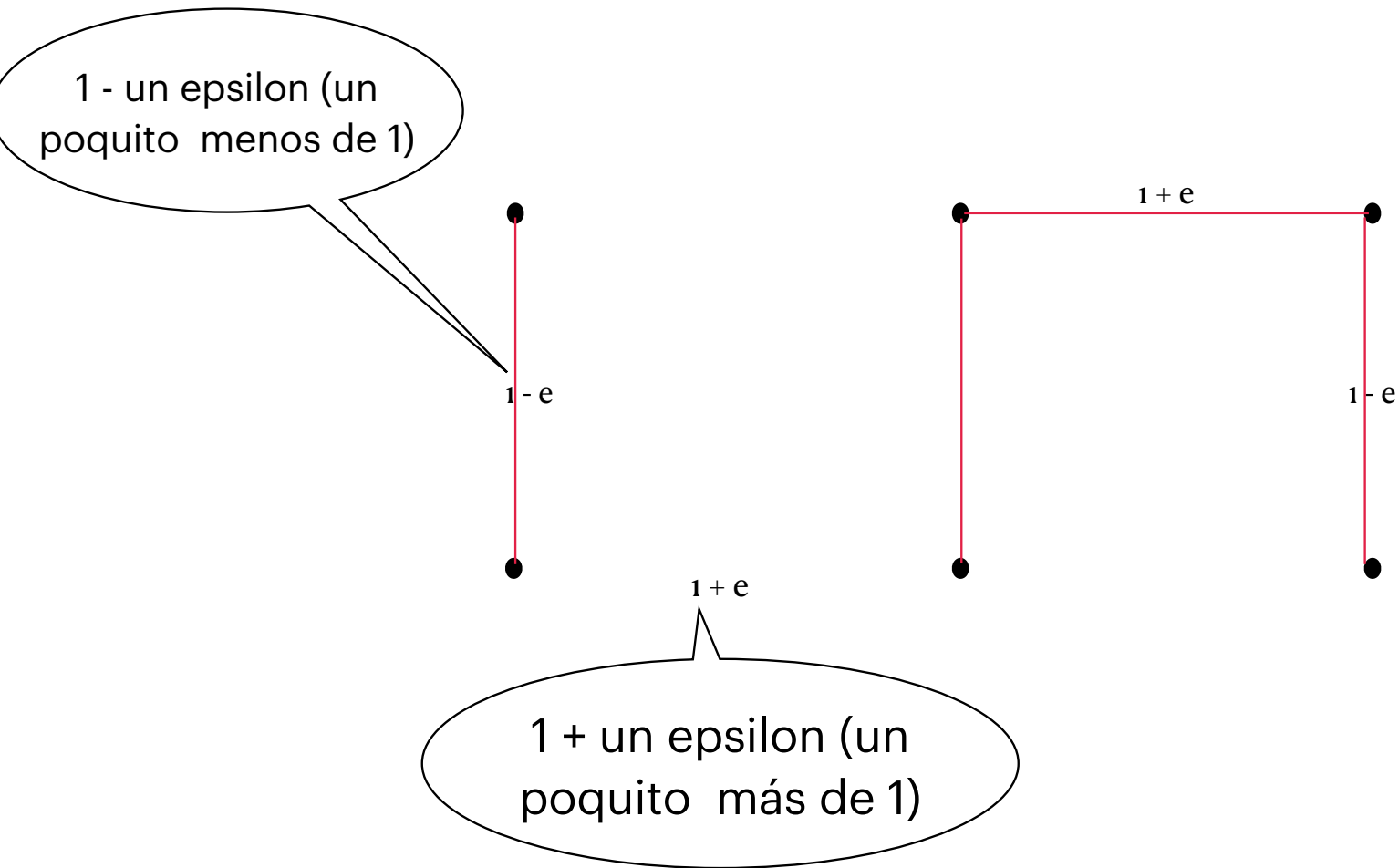
Instancia de Pares más Próximos



Instancia de Pares más Próximos

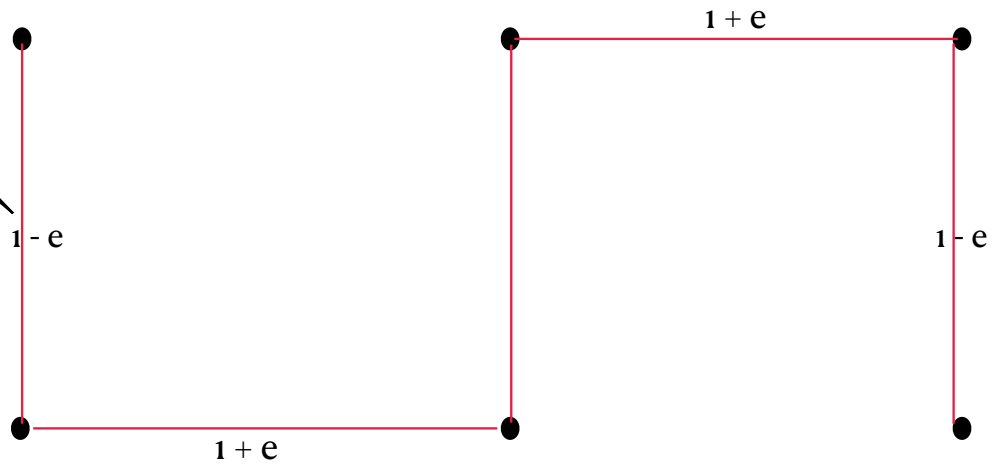


Instancia de Pares más Próximos



Instancia de Pares más Próximos

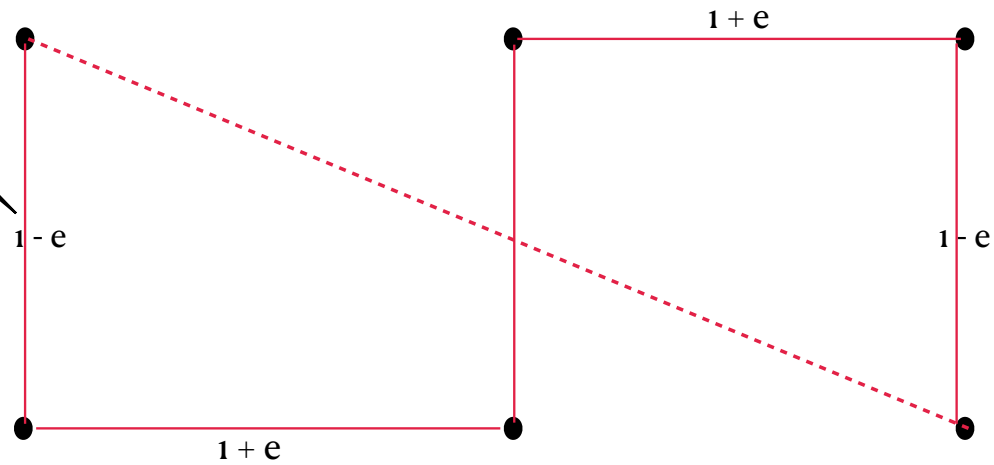
1 - un epsilon (un poquito menos de 1)



1 + un epsilon (un poquito más de 1)

Instancia de Pares más Próximos

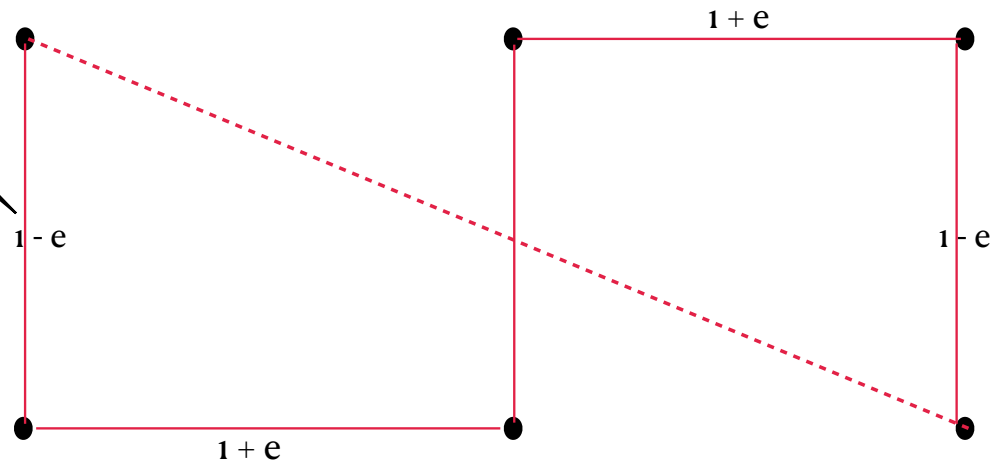
1 - un epsilon (un poquito menos de 1)



1 + un epsilon (un poquito más de 1)

Instancia de Pares más Próximos

1 - un epsilon (un poquito menos de 1)



1 + un epsilon (un poquito más de 1)

Distancia global mínima?

Recorrido Óptimo de Impresión

RecorridoOptimo'(P: Cto. de n puntos en el plano)

```
for i := 1 to n-1 do      n-1 veces  
  begin  
    d := infinity  
    for (s,t) en cadenas de vértices distintas do  
      begin  
        if dist(s,t) <= d then  
          begin  
            d := dist(s,t)  
            sm := s  
            tm := t  
          end  
        end  
      end  
    conectar sm y tm con un arco  
  end  
conectar las dos puntas con un arco
```

Para encontrar el par más cercano, el algoritmo debe examinar todas las parejas posibles de extremos de cadenas.

Si tenemos k extremos libres, el número de combinaciones posibles de pares (sin importar el orden) viene dado por:

$$\binom{k}{2} = \frac{k \cdot (k - 1)}{2} \approx \frac{k^2}{2}$$

$$\sum_{i=1}^{n-1} \frac{(n - i + 1)^2}{2}$$

La cantidad de elementos libres que quedan por revisar en la iteración i (va disminuyendo a medida que i crece)

$$O(n^3)$$

Usando las estructuras de datos correctas podemos mejorarlo!.

Recorrido Óptimo de Impresión

Segunda Propuesta

RecorridoOptimo'(P: Cto. de n puntos en el plano)

```
for i := 1 to n-1 do      n-1 veces  
  begin  
    d := infinity  
    for (s,t) en cadenas de vértices distintas do  
      begin  
        if dist(s,t) <= d then  
          begin  
            d := dist(s,t)  
            sm := s  
            tm := t  
          end  
        end  
      end  
    conectar sm y tm con un arco  
  end  
conectar las dos puntas con un arco
```

Para encontrar el par más cercano, el algoritmo debe examinar todas las parejas posibles de extremos de cadenas.

Si tenemos k extremos libres, el número de combinaciones posibles de pares (sin importar el orden) viene dado por:

$$\binom{k}{2} = \frac{k \cdot (k - 1)}{2} \approx \frac{k^2}{2}$$

$$\sum_{i=1}^{n-1} \frac{(n - i + 1)^2}{2}$$

La cantidad de elementos libres que quedan por revisar en la iteración i (va disminuyendo a medida que i crece)

$$O(n^3)$$

Usando las estructuras de datos correctas podemos mejorarlo!.

Recorrido Óptimo de Impresión

Segunda Propuesta

RecorridoOptimo'(P: Cto. de n puntos en el plano)

```
for i := 1 to n-1 do      n-1 veces  
  begin  
    d := infinity  
    for (s,t) en cadenas de vértices distintas do  
      begin  
        if dist(s,t) <= d then  
          begin  
            d := dist(s,t)  
            sm := s  
            tm := t  
          end  
        end  
      end  
    end  
    conectar sm y tm con un arco  
  end  
  conectar las dos puntas con un arco
```

¿Eficiencia?

Para encontrar el par más cercano, el algoritmo debe examinar todas las parejas posibles de extremos de cadenas.

Si tenemos k extremos libres, el número de combinaciones posibles de pares (sin importar el orden) viene dado por:

$$\binom{k}{2} = \frac{k \cdot (k - 1)}{2} \approx \frac{k^2}{2}$$

$$\sum_{i=1}^{n-1} \frac{(n - i + 1)^2}{2}$$

La cantidad de elementos libres que quedan por revisar en la iteración i (va disminuyendo a medida que i crece)

$$O(n^3)$$

Usando las estructuras de datos correctas podemos mejorarlo!.

Recorrido Óptimo de Impresión

Segunda Propuesta

RecorridoOptimo'(P: Cto. de n puntos en el plano)

```
for i := 1 to n-1 do      n-1 veces  
  begin  
    d := infinity  
    for (s,t) en cadenas de vértices distintas do  
      begin  
        if dist(s,t) <= d then  
          begin  
            d := dist(s,t)  
            sm := s  
            tm := t  
          end  
        end  
      end  
    conectar sm y tm con un arco  
  end  
conectar las dos puntas con un arco
```

¿Eficiencia?

Para encontrar el par más cercano, el algoritmo debe examinar todas las parejas posibles de extremos de cadenas.

Si tenemos k extremos libres, el número de combinaciones posibles de pares (sin importar el orden) viene dado por:

$$\binom{k}{2} = \frac{k \cdot (k - 1)}{2} \approx \frac{k^2}{2}$$

$$\sum_{i=1}^{n-1} \frac{(n - i + 1)^2}{2}$$

$$O(n^3)$$

La cantidad de elementos libres que quedan por revisar en la iteración i (va disminuyendo a medida que i crece)

Usando las estructuras de datos correctas podemos mejorarlo!.

Recorrido Óptimo de Impresión

Segunda Propuesta

RecorridoOptimo'(P: Cto. de n puntos en el plano)

for i := 1 **to** n-1 **do**

n-1 veces

begin

d := infinity

for (s,t) en cadenas de vértices distintas **do**

begin

if dist(s,t) <= d **then**

begin

d := dist(s,t)

sm := s

tm := t

end

end

conectar sm y tm con un arco

end

conectar las dos puntas con un arco

¿Eficiencia?

Para encontrar el par más cercano, el algoritmo debe examinar todas las parejas posibles de extremos de cadenas.

Si tenemos k extremos libres, el número de combinaciones posibles de pares (sin importar el orden) viene dado por:

$$\binom{k}{2} = \frac{k \cdot (k - 1)}{2} \approx \frac{k^2}{2}$$

$$\sum_{i=1}^{n-1} \frac{(n - i + 1)^2}{2}$$

La cantidad de elementos libres que quedan por revisar en la iteración i (va disminuyendo a medida que i crece)

$$O(n^3)$$

Usando las estructuras de datos correctas podemos mejorarlo!.

Recorrido Óptimo de Impresión

RecorridoOptimoConHeap(P: Cto. de n puntos en el plano)

c := crear ColaDePrioridad()

for cada par de puntos distintos (u, v) en P do

insertar(c, (u, v), dist(u, v)) // Se guarda el par ordenado por su distancia

Para encontrar el par más cercano, el algoritmo
debe examinar todas las parejas posibles de extremos de cadenas.

for i := 1 to n-1 do

begin

sm := nulo;

tm := nulo

repetir

(s, t) := extraerMinimo(c) $O(\log n)$

hasta que s y t estén en cadenas distintas

conectar s y t con un arco

end

conectar las dos puntas con un arco

$O(n^2 \log n)$

Recorrido Óptimo de Impresión

Segunda Propuesta

RecorridoOptimoConHeap(P: Cto. de n puntos en el plano)

c := crear ColaDePrioridad()

for cada par de puntos distintos (u, v) en P do

insertar(c, (u, v), dist(u, v)) // Se guarda el par ordenado por su distancia

Para encontrar el par más cercano, el algoritmo

debe examinar todas las parejas posibles de extremos de cadenas.

for i := 1 to n-1 do

begin

sm := nulo;

tm := nulo

repetir

(s, t) := extraerMinimo(c) $O(\log n)$

hasta que s y t estén en cadenas distintas

conectar s y t con un arco

end

conectar las dos puntas con un arco

$O(n^2 \log n)$

Recorrido Óptimo de Impresión

Recorrido Óptimo de Impresión

Tercera Propuesta

Recorrido Óptimo de Impresión

Tercera Propuesta

Una tercera propuesta, sin dudas correcta, consiste en **intentar todas las permutaciones**, y quedarse con la mejor.

Recorrido Óptimo de Impresión

Tercera Propuesta

Una tercera propuesta, sin dudas correcta, consiste en **intentar todas las permutaciones**, y quedarse con la mejor.

```
RecorridoOptimo"(P: Cto. de n puntos en el plano)
d := infinity
for cada permutación Pi de P do
  if cost(Pi) <= d then
    begin
      d := cost(Pi)
      Pmin := Pi
    end
  end
end
retornar Pmin
```

Recorrido Óptimo de Impresión

Tercera Propuesta

Una tercera propuesta, sin dudas correcta, consiste en **intentar todas las permutaciones**, y quedarse con la mejor.

```
RecorridoOptimo"(P: Cto. de n puntos en el plano)
  d := infinity
  for cada permutación Pi de P do
    if cost(Pi) <= d then
      begin
        d := cost(Pi)
        Pmin := Pi
      end
    end
  end
  retornar Pmin
```

Desafortunadamente,
esta solución es muy
costosa en tiempo.

Para 20 puntos, tiene
que considerar
2432902008176640000
combinaciones.

Resolviendo el Problema de Recorrido Óptimo

Resolviendo el Problema de Recorrido Óptimo

El problema de determinar un recorrido óptimo ha sido estudiado durante mucho tiempo.

Resolviendo el Problema de Recorrido Óptimo

El problema de determinar un recorrido óptimo ha sido estudiado durante mucho tiempo.

Desafortunadamente, este problema pertenece a la clase de **problemas NP-hard**. Para estos problemas, como veremos más adelante en la materia, las únicas soluciones óptimas conocidas son del tipo “probar todas las posibilidades”.

Resolviendo el Problema de Recorrido Óptimo

El problema de determinar un recorrido óptimo ha sido estudiado durante mucho tiempo.

Desafortunadamente, este problema pertenece a la clase de **problemas NP-hard**. Para estos problemas, como veremos más adelante en la materia, las únicas soluciones óptimas conocidas son del tipo “probar todas las posibilidades”.

En nuestro caso, probar todas las posibilidades consiste en probar con todas las posibles permutaciones de nodos.

Resolviendo el Problema de Recorrido Óptimo

El problema de determinar un recorrido óptimo ha sido estudiado durante mucho tiempo.

Desafortunadamente, este problema pertenece a la clase de **problemas NP-hard**. Para estos problemas, como veremos más adelante en la materia, las únicas soluciones óptimas conocidas son del tipo “probar todas las posibilidades”.

En nuestro caso, probar todas las posibilidades consiste en probar con todas las posibles permutaciones de nodos.

No es difícil darse cuenta que este tipo de procedimiento es computacionalmente costoso, con lo cual sólo es viable para conjuntos pequeños de puntos.

Alternativas a “Probar todas las Posibilidades”

Alternativas a “Probar todas las Posibilidades”

La primera pregunta que deberíamos hacernos es:

Alternativas a “Probar todas las Posibilidades”

La primera pregunta que deberíamos hacernos es:

¿Nuestro programa se usará sólo con conjuntos pequeños?

Alternativas a “Probar todas las Posibilidades”

La primera pregunta que deberíamos hacernos es:

¿Nuestro programa se usará sólo con conjuntos pequeños?

Si es así, podríamos intentar usar directamente el proceso de “probar todas las posibilidades”.

Alternativas a “Probar todas las Posibilidades”

Alternativas a “Probar todas las Posibilidades”

Si esto resulta ser demasiado ineficiente para nuestro caso, tenemos (al menos) las siguientes alternativas:

Alternativas a “Probar todas las Posibilidades”

Si esto resulta ser demasiado ineficiente para nuestro caso, tenemos (al menos) las siguientes alternativas:

- Buscar información adicional sobre el problema en cuestión. Información adicional sobre el dominio del problema podría evitarnos tener que probar todas las posibilidades.

Alternativas a “Probar todas las Posibilidades”

Si esto resulta ser demasiado ineficiente para nuestro caso, tenemos (al menos) las siguientes alternativas:

- Buscar información adicional sobre el problema en cuestión. Información adicional sobre el dominio del problema podría evitarnos tener que probar todas las posibilidades.
- Buscar una solución buena al problema, aunque **no necesariamente óptima**. Dada la complejidad del problema original, podríamos quizás conformarnos si consiguiéramos una solución que eficientemente nos brindara un recorrido próximo al óptimo (aunque no fuera necesariamente mínimo). **Las dos alternativas vistas al comienzo son ejemplos de estas opciones.**

Refinando las Heurísticas para el Recorrido Óptimo

Para conseguir versiones más cercanas a una implementación a partir de las descripciones anteriores para resolver el problema de recorrido óptimo, necesitamos como mínimo resolver los siguientes problemas:

- Decidir cómo representar conjuntos (por ejemplo, mediante AVLs o tablas hash).
- Decidir cómo representar el hecho de que un punto fue visitado
- Implementar la visita de pares de puntos
- Implementar la construcción de permutaciones de una secuencia

Aspectos Importantes en el Diseño de Algoritmos

Aspectos Importantes en el Diseño de Algoritmos

Los ejemplos anteriores ilustran algunos aspectos importantes del diseño de algoritmos, en particular:

Aspectos Importantes en el Diseño de Algoritmos

Los ejemplos anteriores ilustran algunos aspectos importantes del diseño de algoritmos, en particular:

- La importancia de la formalización de problemas

Aspectos Importantes en el Diseño de Algoritmos

Los ejemplos anteriores ilustran algunos aspectos importantes del diseño de algoritmos, en particular:

- La importancia de la formalización de problemas
- La importancia en la elección de los TADs a utilizar para modelar los datos, y las estructuras de datos sobre las cuales éstos se implementan

Aspectos Importantes en el Diseño de Algoritmos

Los ejemplos anteriores ilustran algunos aspectos importantes del diseño de algoritmos, en particular:

- La importancia de la formalización de problemas
- La importancia en la elección de los TADs a utilizar para modelar los datos, y las estructuras de datos sobre las cuales éstos se implementan
- La importancia de conocer clases de problemas (y su complejidad), además de algoritmos asociados a los principales tipos y estructuras de datos

Aspectos Importantes en el Diseño de Algoritmos

Los ejemplos anteriores ilustran algunos aspectos importantes del diseño de algoritmos, en particular:

- La importancia de la formalización de problemas
- La importancia en la elección de los TADs a utilizar para modelar los datos, y las estructuras de datos sobre las cuales éstos se implementan
- La importancia de conocer clases de problemas (y su complejidad), además de algoritmos asociados a los principales tipos y estructuras de datos
- La importancia de la eficiencia de algoritmos, y la relevancia de esto de acuerdo al problema que se desea resolver

Aspectos Importantes en el Diseño de Algoritmos

Los ejemplos anteriores ilustran algunos aspectos importantes del diseño de algoritmos, en particular:

- La importancia de la formalización de problemas
- La importancia en la elección de los TADs a utilizar para modelar los datos, y las estructuras de datos sobre las cuales éstos se implementan
- La importancia de conocer clases de problemas (y su complejidad), además de algoritmos asociados a los principales tipos y estructuras de datos
- La importancia de la eficiencia de algoritmos, y la relevancia de esto de acuerdo al problema que se desea resolver
- La importancia de conocer y elegir técnicas de diseño de algoritmos que nos ayuden a resolver problemas algorítmicamente.